

INSTITUTO DE EDUCACIÓN SUPERIOR TECNOLÓGICA

[COLOCAR NOMBRE DE LA INSTITUCIÓN / SEDE]

PROGRAMA DE ESTUDIOS
TECNOLOGÍAS DE LA INFORMACIÓN

TRABAJO FINAL DE CURSO

PIAD-221 · ALGORITMOS Y PROGRAMACIÓN PARA DESARROLLO DE SOFTWARE

SISTEMA DE GESTIÓN DE INVENTARIO Y
REGISTRO DE VENTAS PARA UNA
MICROEMPRESA

Aplicación de consola desarrollada en lenguaje Python

DATOS DEL TRABAJO	DETALLE
Estudiante	[COLOCAR APELLIDOS Y NOMBRES]
ID / Código de matrícula	[COLOCAR ID]
Programa de estudios	Tecnologías de la Información
Curso	PIAD-221 · Algoritmos y Programación para Desarrollo de Software
Semestre / Ciclo	[COLOCAR SEMESTRE]
Sección / Turno	[COLOCAR SECCIÓN]
Instructor	[COLOCAR NOMBRE DEL INSTRUCTOR]
Empresa del caso práctico	TecnoStore E.I.R.L. · Venta de accesorios tecnológicos
Fecha de presentación	[COLOCAR FECHA]

[COLOCAR CIUDAD] — [COLOCAR AÑO]

ÍNDICE

1. TEMA.....	4
1.1. Delimitación del tema.....	4
2. OBJETIVO.....	4
2.1. Objetivo general.....	4
2.2. Objetivos específicos.....	4
3. CONSIDERACIONES.....	5
3.1. Contenidos curriculares aplicados en la solución.....	5
3.2. Metodología de desarrollo aplicada.....	6
4. CASO PRÁCTICO.....	7
4.1. Presentación del caso.....	7
4.2. Diagnóstico de la situación actual.....	7
4.3. Condiciones para el desarrollo del caso.....	8
4.4. Requisitos funcionales de la propuesta.....	8
4.5. Alcance y limitaciones.....	9
5. PREGUNTAS GUÍA.....	10
5.1. Pregunta 1.....	10
5.2. Pregunta 2.....	13
5.3. Pregunta 3.....	15
5.4. Pregunta 4.....	18
5.5. Pregunta 5.....	21
6. CONSIDERACIONES PARA EL ENTREGABLE.....	23
6.1. Propuesta de solución.....	23
6.1.1. Descripción general.....	23
6.1.2. Arquitectura modular.....	23
6.1.3. Inventario de funciones del sistema.....	24
6.1.4. Reglas de negocio implementadas.....	26
6.1.5. Estructura de archivos del proyecto.....	26
6.2. Esquemas y diagramas de la propuesta.....	28
6.3. Manual de explicación de la arquitectura modular.....	30
6.3.1. Requisitos e instalación.....	30
6.3.2. Puesta en marcha.....	30
6.3.3. Guía de uso del menú.....	30
6.3.4. Convenciones de la interfaz.....	31
6.3.5. Cómo modificar el sistema.....	31
6.3.6. Recuperación ante incidencias.....	31
6.4. Código fuente documentado.....	32
6.4.1. Cabecera y documentación del archivo.....	32
6.4.2. Capa 1 — Constantes de configuración.....	32
6.4.3. Capa 2 — Persistencia con context manager.....	33
6.4.4. Capa 3 — Validación de entradas.....	33
6.4.5. Capa 4 — Lógica de negocio.....	34
6.4.6. Capa 5 — Controlador del menú principal.....	35
6.4.7. Arranque protegido del programa.....	36
6.5. Capturas de pantalla del menú y de la ejecución.....	37
6.6. Pruebas realizadas.....	48

7. NORMAS TÉCNICAS, SEGURIDAD Y SALUD EN EL TRABAJO Y MEDIO AMBIENTE.....	50
7.1. Normas técnicas aplicadas.....	50
7.2. Seguridad y salud en el trabajo.....	50
7.3. Responsabilidad ambiental.....	50
8. CONCLUSIONES.....	52
9. RECOMENDACIONES.....	53
9.1. Para la implementación en la microempresa.....	53
9.2. Para futuras versiones del sistema.....	53
10. REFERENCIAS BIBLIOGRÁFICAS.....	54
ANEXO A. CÓDIGO FUENTE COMPLETO.....	55
ANEXO B. ARCHIVOS DE DATOS GENERADOS.....	72

*Nota: el índice se genera automáticamente al abrir el documento en Microsoft Word. Si no apareciera, haga clic derecho sobre él y elija **Actualizar campos** → **Actualizar toda la tabla**.*

1. TEMA

Sistema de Gestión de Inventario y Registro de Ventas para una Microempresa (Aplicación de Consola en Python).

El trabajo desarrolla una aplicación de consola escrita en Python 3 que automatiza el registro, la consulta y la actualización de los productos de una tienda de accesorios tecnológicos, así como el procesamiento seguro de sus transacciones de venta, garantizando la persistencia de la información en archivos de texto plano.

1.1. Delimitación del tema

ASPECTO	DELIMITACIÓN
Ámbito funcional	Gestión de catálogo de productos (altas, consultas, actualizaciones y bajas), registro de ventas con emisión de comprobante y generación de reportes de inventario y de rendimiento comercial.
Ámbito técnico	Lenguaje Python 3 (versión 3.8 o superior), ejecución en consola (CMD, PowerShell o terminal de Linux/macOS), sin librerías externas ni motores de base de datos.
Persistencia	Archivos de texto plano (.txt) manejados con la sentencia <code>with</code> y los modos de apertura <code>r</code> , <code>w</code> y <code>a</code> .
Usuario final	Personal administrativo de la microempresa, sin conocimientos técnicos avanzados de informática.
Empresa modelo	TecnoStore E.I.R.L., microempresa dedicada a la comercialización de accesorios tecnológicos.

2. OBJETIVO

2.1. Objetivo general

Desarrollar algoritmos y aplicaciones de consola empleando el lenguaje de programación Python y sus estructuras lógicas fundamentales, para la resolución eficiente de problemas computacionales y la gestión automatizada de datos de una microempresa, cumpliendo las normas técnicas, las normas de seguridad y salud en el trabajo y actuando de manera responsable con el medio ambiente.

2.2. Objetivos específicos

- Modelar la información del inventario y de las ventas mediante **estructuras de datos nativas de Python** (listas de diccionarios y diccionarios anidados) que permitan búsquedas, actualizaciones y recorridos eficientes.

- Construir un **menú principal interactivo** controlado por bucles `while` y estructuras condicionales `if-elif-else`, con una opción de salida controlada.
- Aplicar **programación modular**, descomponiendo el sistema en funciones específicas con parámetros definidos, valores de retorno y separación clara entre la lógica de negocio y la interfaz de consola.
- Implementar un **manejo robusto de excepciones** con bloques `try-except-else-finally` que evite el cierre inesperado del programa ante entradas no numéricas, cantidades negativas, identificadores duplicados o errores de lectura y escritura de archivos.
- Garantizar la **persistencia de los datos** entre ejecuciones mediante archivos de texto plano administrados con context managers (`with open(...)`) y los modos de apertura `r`, `w` y `a`.
- Elaborar los **entregables técnicos** del trabajo: archivo `.py` funcional, código documentado, capturas de pantalla de la ejecución, diagramas de la solución y el manual de la arquitectura modular.

3. CONSIDERACIONES

El presente trabajo final consiste en resolver el caso práctico presentado, utilizando como referencia el problema planteado y las preguntas guía proporcionadas para orientar el desarrollo de la propuesta.

La propuesta se fundamenta en los conocimientos adquiridos a lo largo del curso, aplicando lo aprendido en las tareas y operaciones descritas en los contenidos curriculares. La siguiente tabla relaciona cada contenido del curso con la evidencia concreta de su aplicación dentro de la solución desarrollada.

3.1. Contenidos curriculares aplicados en la solución

CONTENIDO DEL CURSO	APLICACIÓN EN EL TRABAJO FINAL	EVIDENCIA
Tipos de datos, variables y operadores	Manejo de cadenas (código y nombre), enteros (stock y cantidades) y flotantes (precios, descuentos, IGV y totales) con conversión explícita y redondeo a dos decimales.	Funciones <code>crear_producto()</code> y <code>calcular_importe_item()</code>
Estructuras condicionales	Derivación del menú con <code>if-elif-else</code> , clasificación del estado del stock (OK, CRÍTICO, AGOTADO) y validación de rangos.	Función <code>main()</code> y <code>mostrar_tabla_productos()</code>
Estructuras repetitivas	Bucle <code>while</code> del menú persistente y de la carga de ítems de la venta; recorridos <code>for</code> sobre el inventario y el historial; uso de <code>break</code> y <code>continue</code> .	Funciones <code>main()</code> y <code>opcion_registrar_venta()</code>
Estructuras de datos	Listas de diccionarios para el inventario, diccionario anidado para cada venta y comprensión de listas para los filtros de los reportes.	Figura C (modelo de datos)

CONTENIDO DEL CURSO	APLICACIÓN EN EL TRABAJO FINAL	EVIDENCIA
Programación modular	Cincuenta y un funciones organizadas en cinco capas, con parámetros, valores de retorno y responsabilidad única.	Figura A (arquitectura modular)
Manejo de excepciones	Bloques try-except-else-finally para ValueError, FileNotFoundError, PermissionError, KeyError, UnicodeDecodeError y KeyboardInterrupt.	Figura D y archivo datos/errores.log
Manejo de archivos	Lectura y escritura de archivos de texto con with open(...) en los modos r, w y a; métodos read(), readlines() y write().	Capa 2 de persistencia y Figura 16
Documentación de software	Docstrings en las 47 funciones, comentarios explicativos por bloque y manual de arquitectura modular.	Anexo A (código fuente)

3.2. Metodología de desarrollo aplicada

El desarrollo se realizó en cinco etapas sucesivas, propias del ciclo de vida básico del software:

ETAPA	ACTIVIDAD REALIZADA	RESULTADO OBTENIDO
1. Análisis	Lectura del caso, identificación de los problemas de la gestión manual y determinación de los requerimientos funcionales.	Cuadro de diagnóstico (sección 4.2)
2. Diseño	Definición del modelo de datos, de la arquitectura modular por capas y de los diagramas de flujo del sistema.	Figuras A, B, C, D y E
3. Codificación	Programación del archivo sistema_inventario.py aplicando programación modular y documentación mediante docstrings.	Archivo .py funcional (Anexo A)
4. Pruebas	Ejecución de casos de prueba con datos válidos e inválidos para verificar la validación de entradas y la persistencia.	Cuadro de pruebas (sección 6.6)
5. Documentación	Elaboración del manual de arquitectura modular y registro de las capturas de pantalla de la ejecución.	Secciones 6.3 y 6.5

4. CASO PRÁCTICO

4.1. Presentación del caso

Una pequeña tienda de accesorios tecnológicos lleva actualmente sus registros de stock y ventas en cuadernos y hojas de cálculo desconectadas, lo que ocasiona desactualización constante del inventario, errores humanos al calcular precios o descuentos, y pérdida de tiempo en la búsqueda de productos.

Esta gestión manual impide conocer el rendimiento diario real, genera discrepancias entre el stock físico y el registrado, y limita la capacidad de toma de decisiones del administrador, afectando la rentabilidad y la atención al cliente.

La empresa requiere el desarrollo de una **aplicación de consola programada en Python** que automatice el registro, consulta y actualización de productos en inventario, así como el procesamiento seguro de transacciones de venta. La solución debe permitir cargar y guardar la información en archivos de texto para garantizar la persistencia de datos, validar rigurosamente las entradas del usuario para evitar cierres inesperados del programa, y ofrecer un menú interactivo intuitivo que pueda ser operado por personal sin conocimientos técnicos avanzados.

4.2. Diagnóstico de la situación actual

Del enunciado del caso se desprenden cinco problemas centrales. Para cada uno se identificó su causa raíz, el efecto que produce en la microempresa y la solución informática incorporada en la propuesta:

PROBLEMA IDENTIFICADO	CAUSA	EFEECTO EN LA EMPRESA	SOLUCIÓN IMPLEMENTADA
Desactualización constante del inventario	Los registros se anotan en cuadernos y hojas de cálculo desconectadas entre sí.	Diferencias entre el stock físico y el registrado; ventas de productos agotados.	El stock se descuenta automáticamente al registrar la venta y se graba de inmediato en <code>inventario.txt</code> .
Errores humanos en los cálculos	Precios, descuentos e impuestos se calculan manualmente.	Cobros incorrectos, pérdida de margen y reclamos de clientes.	Funciones <code>calcular_descuento()</code> , <code>calcular_importe_item()</code> y <code>construir_venta()</code> calculan descuento por volumen, IGV y total.
Pérdida de tiempo en la búsqueda de productos	No existe un catálogo consultable de forma inmediata.	Atención lenta al cliente y colas en el mostrador.	Opción 3 del menú: búsqueda por código exacto o por texto libre en nombre y categoría.
Desconocimiento del rendimiento diario	No se consolidan las ventas del día.	El administrador no cuenta con información para decidir compras ni promociones.	Opciones 7 y 8: reporte de inventario valorizado y reporte de ventas con ticket promedio y ranking de productos.
Riesgo de pérdida de información	Los datos dependen de documentos físicos o archivos dispersos.	Información irrecuperable ante extravío o deterioro.	Persistencia automática en archivos de texto plano tras cada operación y bitácora de incidencias.

4.3. Condiciones para el desarrollo del caso

El enunciado establece tres condiciones obligatorias. La siguiente tabla demuestra cómo se cumple cada una dentro del archivo `sistema_inventario.py`:

CONDICIÓN EXIGIDA	CUMPLIMIENTO EN LA SOLUCIÓN	EVIDENCIA
1. Programación modular , definiendo funciones específicas para cada operación (carga de datos, alta de productos, registro de ventas, generación de reportes y guardado en archivo), evitando la programación lineal o monolítica.	El sistema está compuesto por 51 funciones distribuidas en cinco capas. Cada operación exigida tiene su función propia: carga de datos (<code>cargar_inventario()</code> , <code>cargar_ventas()</code>), alta de productos (<code>crear_producto()</code> y <code>agregar_producto()</code>), registro de ventas (<code>crear_item_venta()</code> , <code>construir_venta()</code>), reportes (<code>resumen_ventas()</code> , <code>ranking_productos()</code> , <code>valorizar_inventario()</code>) y guardado (<code>guardar_inventario()</code> , <code>registrar_venta_en_archivo()</code>). Ninguna instrucción ejecutable se encuentra fuera de una función, salvo la llamada a <code>main()</code> .	Figura A · sección 6.1.2
2. Manejo robusto de excepciones (<code>try-except-else-finally</code>) para validar tipos de datos (evitar entradas no numéricas, cantidades negativas o IDs duplicados) y capturar errores de lectura/escritura de archivos sin interrumpir la ejecución.	Se emplean 20 bloques <code>try</code> (32 cláusulas <code>except</code> y 9 bloques <code>finally</code>), con la estructura completa de cuatro cláusulas en las funciones críticas. Se capturan <code>ValueError</code> (entradas no numéricas), <code>FileNotFoundError</code> , <code>PermissionError</code> , <code>OSError</code> , <code>UnicodeDecodeError</code> , <code>KeyError</code> y <code>KeyboardInterrupt</code> . Las cantidades negativas se rechazan por rango y los IDs duplicados mediante <code>existe_id()</code> . Toda excepción se registra en <code>datos/errores.log</code> y la ejecución continúa.	Figura D · Figuras 4 y 8
3. Persistencia exclusiva en archivos de texto plano utilizando la sentencia <code>with</code> (context managers) y los modos de apertura adecuados (<code>r</code> , <code>w</code> , <code>a</code>), asegurando que los datos se conserven entre ejecuciones.	Las seis operaciones de disco del sistema se realizan dentro de un <code>with open(...)</code> . Se usa <code>r</code> para cargar, <code>w</code> para reescribir el inventario actualizado y <code>a</code> para agregar ventas e incidencias. No se emplea ninguna base de datos ni formato binario. La Figura 15 demuestra la recuperación de los datos en una segunda ejecución.	Figura C · Figuras 15 y 16

4.4. Requisitos funcionales de la propuesta

El caso práctico exige que la propuesta incluya seis elementos. Todos fueron implementados y verificados:

REQUISITO SOLICITADO	IMPLEMENTACIÓN	ESTADO
Menú principal interactivo controlado por bucles (<code>while</code>) y estructura condicional (<code>if-elif-else</code>) con opción de salida.	Función <code>main()</code> : bucle <code>while True</code> con derivación <code>if-elif-else</code> de diez opciones y salida controlada mediante <code>break</code> en la opción 0.	CUMPLE
Uso de estructuras de datos nativas de Python (listas de diccionarios o diccionarios anidados) para almacenar productos, precios, stock y registros de ventas.	Inventario: lista de diccionarios. Venta: diccionario anidado con lista interna de ítems. Ranking: diccionario acumulador.	CUMPLE
Funciones modulares con parámetros definidos, valores de retorno y separación clara de responsabilidades (lógica de negocio vs. interfaz de consola).	Las 19 funciones de la capa de negocio no contienen <code>print()</code> ni <code>input()</code> ; reciben parámetros y devuelven valores. La interfaz reside en una capa independiente.	CUMPLE
Validación de entrada de datos y bloqueos <code>try-except</code> para manejar <code>ValueError</code> , <code>FileNotFoundError</code> y	Capa 3 de validación (<code>leer_entero()</code> , <code>leer_decimal()</code> , <code>leer_cadena()</code> , <code>leer_opcion()</code>) más el tratamiento de registros corruptos línea por línea en la carga de	CUMPLE

REQUISITO SOLICITADO	IMPLEMENTACIÓN	ESTADO
errores de formato.	archivos.	
Lectura y escritura de archivos .txt o .csv mediante <code>with open(...)</code> y métodos <code>read()</code> , <code>write()</code> , <code>append()</code> .	Archivos .txt delimitados por punto y coma. Métodos <code>read()</code> , <code>readlines()</code> y <code>write()</code> ; el modo <code>a</code> (<code>append</code>) se emplea para el historial de ventas y la bitácora.	CUMPLE
Entregable: archivo .py funcional, código documentado (con capturas de pantalla del menú y de la ejecución) y breve manual de explicación de la arquitectura modular.	Archivo <code>sistema_inventario.py</code> (1 538 líneas, 51 funciones con <code>docstring</code>), 16 capturas de pantalla de la ejecución real (sección 6.5) y manual de arquitectura modular (sección 6.3).	CUMPLE

4.5. Alcance y limitaciones

EL SISTEMA SÍ CONTEMPLA	EL SISTEMA NO CONTEMPLA
Registro, consulta, actualización y baja de productos del catálogo.	Gestión de compras a proveedores y cuentas por pagar.
Registro de ventas con varios productos por comprobante, descuento por volumen e IGV.	Facturación electrónica con envío a la administración tributaria.
Reportes de inventario valorizado, stock crítico, ventas del día y ranking de productos.	Gráficos estadísticos o exportación a formatos ofimáticos.
Persistencia en archivos de texto plano y bitácora de incidencias.	Motores de base de datos, acceso concurrente o uso en red.
Operación por un usuario a la vez en una terminal local.	Control de usuarios, contraseñas y perfiles de acceso.

Estas limitaciones son coherentes con el alcance del curso, que se centra en algoritmos, estructuras lógicas fundamentales, programación modular, manejo de excepciones y archivos de texto, y se recogen como líneas de mejora en la sección de recomendaciones.

5. PREGUNTAS GUÍA

El desarrollo de las preguntas guía tiene el propósito de orientar la generación de la propuesta sobre el caso práctico. A continuación se responde cada una fundamentando la decisión técnica adoptada y mostrando el fragmento del programa donde se aplica.

5.1. Pregunta 1

¿Cómo se pueden modelar los datos del inventario y las ventas utilizando listas y diccionarios para permitir búsquedas, actualizaciones y recorridos eficientes?

a) Modelo adoptado

El inventario se modela como una **lista de diccionarios**. La lista aporta el orden y la posibilidad de recorrer el catálogo de forma secuencial con `for`; el diccionario aporta el acceso a cada atributo por medio de una clave con significado (`producto["stock"]`) en lugar de una posición numérica sin sentido semántico (`producto[4]`). Cada producto es, entonces, un registro autocontenido:

```
producto = {"id": "P001", "nombre": "Audifonos Bluetooth ANC",
            "categoria": "Audio", "precio": 159.90, "stock": 14}

inventario = [producto_1, producto_2, ..., producto_n] # lista de diccionarios
```

Estructura del inventario en memoria (capa de lógica de negocio).

Las ventas, en cambio, responden a una relación **uno a muchos**: un comprobante contiene varios productos. Por ello se modelan como un **diccionario anidado** cuya clave `items` almacena una lista interna de diccionarios, reproduciendo con exactitud la estructura de una boleta real (cabecera, detalle y totales):

```
venta = {"boleta": "B0001", "fecha": "12/09/2026 00:00:06",
        "cliente": "Maria Quispe Ruiz",
        "items": [{"id": "P002", "cantidad": 6, "precio_unitario": 49.90,
                    "descuento": 8.0, "importe": 275.45},
                  {"id": "P004", "cantidad": 12, ...}],
        "subtotal": 839.83, "igv": 151.17, "total": 991.00}
```

Estructura de una venta: diccionario con una lista anidada de ítems.

b) Por qué esta combinación es eficiente

OPERACIÓN	MECANISMO APLICADO	FUNCIÓN DEL SISTEMA
Búsqueda	Recorrido <code>for</code> sobre la lista comparando la clave <code>id</code> ; la función retorna la referencia al diccionario, no una copia.	<code>buscar_producto_por_id()</code>
Búsqueda parcial	Operador <code>in</code> sobre las cadenas <code>nombre</code> y <code>categoria</code> convertidas a minúsculas, acumulando coincidencias en una lista nueva.	<code>buscar_productos_por_texto()</code>
Actualización	Los diccionarios son mutables : al modificar la referencia obtenida en la búsqueda, el cambio se	<code>actualizar_stock()</code> · <code>actualizar_precio()</code>

OPERACIÓN	MECANISMO APLICADO	FUNCIÓN DEL SISTEMA
	refleja de inmediato en la lista, sin reconstruirla.	
Alta y baja	Métodos nativos <code>append()</code> y <code>remove()</code> de la lista, previa verificación de duplicados.	<code>agregar_producto()</code> · <code>eliminar_producto()</code>
Recorrido y filtrado	Comprensión de listas para obtener subconjuntos (stock crítico, productos disponibles, productos por categoría).	<code>productos_stock_critico()</code>
Totalizaciones	Función <code>sum()</code> con expresiones generadoras, que no crean listas intermedias en memoria.	<code>valorizar_inventario()</code> · <code>resumen_ventas()</code>
Acumulación	Diccionario acumulador <code>{id: unidades}</code> con el método <code>get()</code> , y ordenamiento con <code>sorted()</code> .	<code>ranking_productos()</code>

```
def buscar_producto_por_id(inventario, id_producto):
    """Busca un producto por su identificador unico."""
    if id_producto is None:
        return None
    clave = id_producto.strip().upper()
    for producto in inventario:
        if producto["id"] == clave:
            return producto
    return None
```

Búsqueda en la lista de diccionarios: el valor de retorno permite actualizar el producto directamente.

c) Alternativas descartadas

- **Listas paralelas** (una lista para los códigos, otra para los precios, otra para el stock): obligan a mantener sincronizados los índices; un `remove()` en una de ellas desalinea todas las demás y produce datos cruzados.
- **Cadenas de texto con delimitadores en memoria**: exigirían fragmentar la cadena en cada consulta, con conversiones repetidas y alto riesgo de error de formato.
- **Diccionario único indexado por ID** (`{"P001": {...}}`): ofrece búsqueda directa $O(1)$, pero pierde el orden de captura del catálogo y complica los recorridos ordenados de los reportes. Para el volumen de una microempresa —decenas o pocos cientos de productos— la búsqueda lineal resulta imperceptible; no obstante, esta estructura queda propuesta como mejora en la sección de recomendaciones.

Figura C. Modelo de datos y esquema de persistencia en archivos de texto
Estructuras nativas de Python empleadas: **lista de diccionarios** para el inventario y **diccionario anidado** (con lista interna de ítems) para cada venta.



Figura C. Modelo de datos y esquema de persistencia en archivos de texto.

5.2. Pregunta 2

¿Qué combinación de estructuras de control (`if-elif-else`, `while`, `for`, `break`, `continue`) es la más adecuada para construir un menú persistente que permita al usuario navegar entre opciones hasta decidir salir?

a) Combinación adoptada

La combinación más adecuada es un **bucle `while True` que contiene una estructura `if-elif-else` y del cual se sale únicamente mediante `break`**. El bucle mantiene el menú activo de forma indefinida, la estructura condicional deriva la ejecución hacia la función especializada de cada opción y **break** proporciona un único punto de salida controlado, en el que se guarda la información antes de terminar.

```
while True:                                # menu persistente
    limpiar_pantalla()
    mostrar_menu_principal()
    opcion = leer_opcion(" Seleccione una opcion [0-9]: ", OPCIONES_MENU)

    if opcion == "1":
        opcion_listar_inventario(inventario)
    elif opcion == "2":
        opcion_registrar_producto(inventario)
    elif opcion == "9":
        opcion_guardar_cambios(inventario)
    else:
        guardar_inventario(inventario)      # opcion "0"
        print(" Gracias por utilizar el sistema. Hasta pronto!") # guardado de seguridad
        break                               # UNICA salida del bucle

pausar()
```

Estructura del menú principal en la función main() (capa de interfaz y control).

b) Función de cada estructura dentro del sistema

ESTRUCTURA	DÓNDE SE APLICA	JUSTIFICACIÓN TÉCNICA
while True	Bucle del menú principal y bucle de carga de ítems de una venta.	El número de operaciones que realizará el usuario es desconocido de antemano; el bucle debe repetirse mientras el operador lo decida, no una cantidad fija de veces.
if-elif-else	Derivación de las diez opciones del menú y clasificación del estado del stock.	Las opciones son mutuamente excluyentes: en cuanto una condición se cumple, las demás se descartan, lo que evita comparaciones innecesarias. El <code>else</code> final captura la opción de salida.
for	Recorrido del inventario, del historial de ventas y de los ítems de un comprobante.	El número de elementos es conocido y finito: se recorre la colección completa sin necesidad de administrar manualmente un contador ni el índice.
break	Salida del menú (opción 0) y fin de la carga de ítems (tecla F).	Ofrece un punto de término único y explícito. Se prefiere a una variable bandera porque evita evaluar una condición adicional en cada vuelta y hace evidente dónde termina el bucle.
continue	Reintento tras una entrada inválida, un producto inexistente o stock insuficiente.	Descarta la iteración defectuosa y vuelve a solicitar el dato sin abandonar la operación en curso: el usuario no pierde lo ya digitado.

c) Alternativas descartadas

- **`while opcion != "0"` con lectura previa:** obliga a duplicar la instrucción de lectura (una antes del bucle y otra al final) y, si se olvida la segunda, produce un bucle infinito.
- **Variable bandera (`continuar = True`):** añade una variable de estado que puede quedar desactualizada en alguna rama del condicional; `break` expresa la misma intención de forma más directa.
- **Menú recursivo (la función se llama a sí misma):** cada opción elegida consumiría un nivel de la pila de llamadas y, tras un uso prolongado, provocaría un error de recursión.

Figura B. Diagrama de flujo del programa principal — función `main()`

Menú persistente controlado por un bucle `while` y una estructura condicional `if - elif - else` con opción de salida.

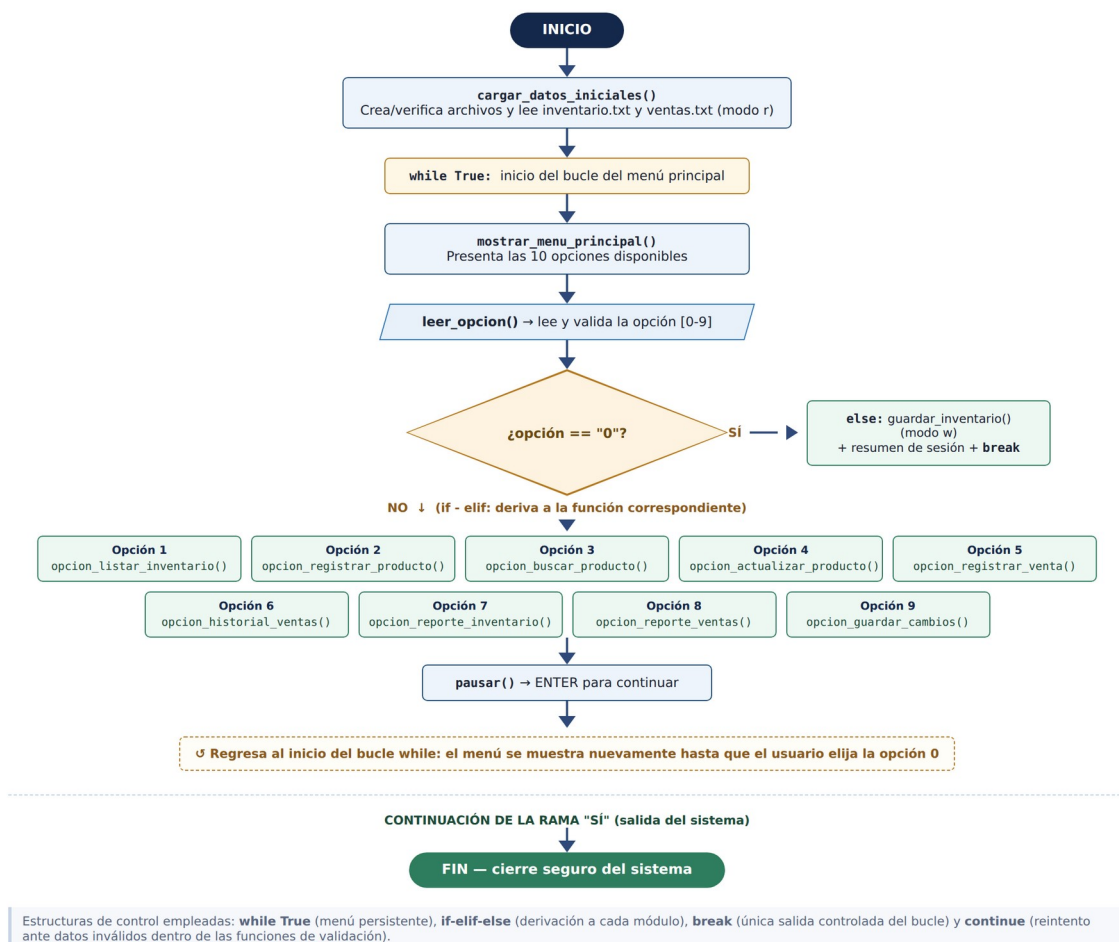


Figura B. Diagrama de flujo del programa principal: bucle `while`, derivación `if-elif-else` y salida con `break`.

5.3. Pregunta 3

¿De qué manera se deben descomponer las operaciones del sistema en funciones reutilizables, definiendo correctamente los parámetros, el ámbito de las variables (local/global) y los valores de retorno?

a) Criterio de descomposición

La descomposición se rige por el principio de **responsabilidad única**: cada función resuelve una sola tarea, su nombre la describe con un verbo en infinitivo y su extensión no supera lo que puede leerse de una sola vez. Aplicando este criterio, el sistema quedó dividido en 51 funciones agrupadas en cinco capas, según el siguiente reparto:

CAPA	RESPONSABILIDAD	N.º DE FUNCIONES	¿USA print / input?
1. Configuración	Constantes globales del sistema (no contiene funciones, sino 9 constantes).	—	No
2. Persistencia	Lectura y escritura en archivos de texto; serialización y deserialización de registros.	10	Solo mensajes de estado
3. Validación	Entrada segura de datos desde el teclado.	5	Sí (es su propósito)
4. Lógica de negocio	Reglas del negocio: búsquedas, altas, cálculos, reportes.	19	No (capa pura)
5. Interfaz y control	Presentación en consola y coordinación del menú.	17	Sí

b) Definición de parámetros

Los parámetros se definen de modo que cada función reciba **todo lo que necesita** y no dependa de variables externas. Las rutas de archivo se declaran como parámetros con valor por omisión, de manera que la función funciona sin argumentos en la operación normal y, al mismo tiempo, puede apuntarse a otro archivo durante las pruebas sin modificar su código:

```
def guardar_inventario(inventario, ruta=ARCHIVO_INVENTARIO):  
    """Graba TODO el inventario en el archivo de texto (modo "w")."""  
  
def leer_entero(mensaje, minimo=None, maximo=None, cancelable=True):  
    """Lee un numero entero validando tipo de dato y rango permitido."""
```

Parámetros obligatorios seguidos de parámetros con valor por omisión (reutilización sin duplicar código).

c) Ámbito de las variables

ÁMBITO	QUÉ SE ALMACENA	REGLA APLICADA EN EL SISTEMA
Global	Constantes de configuración: IGV, STOCK_MINIMO, ARCHIVO_INVENTARIO, CATEGORIAS, ESCALA_DESCUENTOS.	Se escriben en MAYÚSCULAS y se usan solo en modo lectura . En todo el programa no se emplea la palabra reservada <code>global</code> , de modo que ninguna función puede alterar el estado general de forma imprevista.
Local	Variables de trabajo: producto, items, resultante, opcion, numero.	Nacen y mueren dentro de la función. Al no existir fuera de ella, dos funciones pueden usar el mismo nombre sin interferirse.
Parámetros	Las listas inventario y ventas.	Al ser objetos mutables, se pasan por

ÁMBITO	QUÉ SE ALMACENA	REGLA APLICADA EN EL SISTEMA
(referencia)		referencia: la función que las recibe puede modificarlas y el cambio es visible para quien la llamó. Por ello cada docstring declara si la función modifica o no la estructura recibida.

d) Valores de retorno

Ninguna función de la capa de negocio imprime su resultado: lo **devuelve**. Esto permite que la misma función sirva a la consola de hoy y a una futura interfaz gráfica o web sin modificar una sola línea. Los tipos de retorno se normalizaron así:

TIPO DE RETORNO	SIGNIFICADO	EJEMPLOS
bool	Éxito o fracaso de una operación.	guardar_inventario() · agregar_producto() · confirmar()
dict	Un registro completo (producto, ítem o comprobante).	crear_producto() · crear_item_venta() · construir_venta()
list	Un conjunto de registros, vacío si no hay coincidencias.	cargar_inventario() · buscar_productos_por_texto()
int / float	Un valor calculado.	actualizar_stock() · valorizar_inventario() · calcular_descuento()
None	Ausencia de resultado o cancelación del usuario.	buscar_producto_por_id() cuando el código no existe
Excepción	Condición inválida que quien llama debe resolver.	agregar_producto() lanza ValueError ante un ID duplicado

El siguiente ejemplo resume las tres ideas: parámetros explícitos, ausencia de variables globales modificables y comunicación por valor de retorno y por excepción.

```
def actualizar_stock(inventario, id_producto, movimiento):
    """Aplica un movimiento de stock (positivo ingreso, negativo salida).

    Retorna: int -> stock resultante despues del movimiento.
    Lanza : KeyError si el producto no existe.
           ValueError si el movimiento dejaria el stock en negativo.
    """
    producto = buscar_producto_por_id(inventario, id_producto) # reutilizacion
    if producto is None:
        raise KeyError("Producto no encontrado: " + str(id_producto))
    resultante = producto["stock"] + int(movimiento) # variable local
    if resultante < 0:
        raise ValueError("Stock insuficiente: disponible {0}, solicitado {1}".format(
            producto["stock"], abs(movimiento)))
    producto["stock"] = resultante # muta por referencia
    return resultante # valor de retorno
```

La misma función atiende la venta (movimiento negativo) y el ingreso de mercadería (movimiento positivo): una sola función, dos usos.



CAPA 5

INTERFAZ DE CONSOLA Y CONTROL

17 funciones

Diálogo con el usuario y coordinación del menú principal (while + if-elif-else).

main()

cargar_datos_iniciales()

mostrar_encabezado()

mostrar_menu_principal()

mostrar_tabla_productos()

mostrar_comprobante()

opcion_listar_inventario()

opcion_registrar_producto()

opcion_buscar_producto()

opcion_actualizar_producto()

opcion_registrar_venta()

opcion_historial_ventas()

opcion_reporte_inventario()

opcion_reporte_ventas()

opcion_guardar_cambios()

limpiar_pantalla()

pausar()

Flujo de llamadas

CAPA 5 → CAPA 3 (pedir datos) → CAPA 4 (calcular) → CAPA 2 (grabar). Nunca en sentido inverso.

Ventaja

Un cambio en la presentación no obliga a modificar las reglas del negocio ni el acceso a archivos.

Ámbito de variables

Las constantes son globales; el inventario y las ventas viajan como parámetros locales.

Archivos generados por la CAPA 2: **datos/inventario.txt** (modo w), **datos/ventas.txt** (modo a) y **datos/errores.log** (modo a).

Figura A. Arquitectura modular: las 51 funciones del sistema distribuidas en cinco capas.

5.4. Pregunta 4

¿Qué errores comunes pueden ocurrir durante la interacción con el usuario o la manipulación de archivos, y cómo se estructuran los bloques `try-except-else-finally` para garantizar la estabilidad del programa?

a) Errores previstos y su tratamiento

El sistema será operado por personal administrativo sin formación técnica, por lo que se anticiparon los errores más frecuentes en este escenario. Cada uno se captura de forma **específica**, nunca con un `except` genérico que oculte la causa real:

EXCEPCIÓN	SITUACIÓN QUE LA PROVOCA	TRATAMIENTO EN EL SISTEMA
<code>ValueError</code>	El operador escribe "seis" donde se pide una cantidad, o "ciento ochenta" en el precio.	Se informa el dato inválido, se registra en la bitácora y se vuelve a solicitar mediante <code>continue</code> . El programa no se detiene (Figuras 4 y 8).
<code>FileNotFoundError</code>	Primera ejecución del sistema: aún no existen <code>inventario.txt</code> ni <code>ventas.txt</code> .	Se avisa la situación y se arranca con estructuras vacías; los archivos se crean en el primer guardado.
<code>PermissionError</code>	El archivo está abierto en otro programa o la carpeta es de solo lectura.	Se notifica al operador, se registra el detalle y el menú continúa operativo.
<code>OSError</code>	Disco lleno, unidad extraíble retirada o ruta inválida.	Se captura en todas las operaciones de escritura y la función devuelve <code>False</code> .
<code>UnicodeDecodeError</code>	El archivo de datos fue editado manualmente y quedó con otra codificación.	Se descarta la lectura, se registra el error y el sistema continúa.
<code>KeyError</code>	Se intenta actualizar o vender un producto dado de baja del catálogo.	Se captura en la capa de interfaz y se muestra el mensaje "Producto no encontrado".
<code>EEOFError</code> · <code>KeyboardInterrupt</code>	El usuario presiona <code>Ctrl+C</code> o <code>Ctrl+Z</code> mientras digita un dato.	Se cancela la operación en curso y se realiza un cierre seguro guardando el inventario.
Excepción no prevista	Cualquier fallo no contemplado en tiempo de ejecución.	Red de seguridad final en <code>if __name__ == "__main__":</code> se registra el detalle técnico y se cierra ordenadamente.

b) Estructura de los bloques

La estructura completa `try-except-else-finally` reparte la responsabilidad en cuatro partes bien delimitadas:

BLOQUE	QUÉ CONTIENE	CUÁNDO SE EJECUTA
<code>try</code>	Únicamente la instrucción que puede fallar: la conversión de la entrada o la apertura del archivo. Mantenerlo corto evita capturar errores que no se esperaban.	Siempre (es el intento).
<code>except</code>	El mensaje comprensible para el operador, el registro en <code>datos/errores.log</code> y la decisión de reintentar (<code>continue</code>) o abandonar la operación.	Solo si se produce la excepción indicada.
<code>else</code>	La validación de rango y la confirmación de la operación (por ejemplo, grabar el inventario o agregar el ítem a la venta).	Solo si no hubo excepción.
<code>finally</code>	El mensaje de cierre de la operación y la liberación de	Siempre, haya error o no,

BLOQUE	QUÉ CONTIENE	CUÁNDO SE EJECUTA
	recursos.	incluso si la función retorna dentro del try.

```
def guardar_inventario(inventario, ruta=ARCHIVO_INVENTARIO):
    grabado = False
    try:
        with open(ruta, "w", encoding="utf-8") as archivo:    # puede fallar
            for producto in inventario:
                archivo.write(producto_a_linea(producto))
    except PermissionError as error:    # archivo abierto o protegido
        print(" [!] No se pudo grabar el inventario (permisos):", error)
        registrar_error("guardar_inventario", str(error))
    except OSError as error:    # disco lleno, ruta invalida
        print(" [!] Error de escritura en disco:", error)
        registrar_error("guardar_inventario", str(error))
    else:    # no hubo excepcion
        grabado = True
        print(" [OK] Inventario grabado en '{0}'".format(ruta))
    finally:    # pase lo que pase
        print(" [i] Operacion de guardado finalizada.")
    return grabado
```

Estructura completa try-except-else-finally en la capa de persistencia.

c) Criterios de estabilidad aplicados

- **Validar antes, capturar después:** los errores previsibles (rango, duplicados, stock) se impiden con condiciones; los imprevisibles (archivo, codificación, interrupción) se capturan con `except`.
- **Nunca dejar el sistema en estado inconsistente:** en una venta, el stock se descuenta solo después de que el comprobante se construyó correctamente; si algo falla antes, el inventario permanece intacto.
- **Errores acotados:** un registro corrupto dentro de `inventario.txt` se descarta individualmente con `continue` y el resto del catálogo se carga con normalidad, en lugar de abortar la lectura completa.
- **Trazabilidad:** toda excepción capturada se escribe en `datos/errores.log` con fecha, función de origen y detalle técnico, lo que permite al instructor o al administrador auditar lo ocurrido (Figura 16).
- **Salida siempre controlada:** la aplicación no termina nunca con un mensaje de error de Python (`*traceback*`); incluso ante un fallo imprevisto se muestra un aviso legible y se cierra con código de salida 0.

Figura D. Esquema del manejo de excepciones try - except - else - finally

Estrategia aplicada para que ninguna entrada del usuario ni error de archivo provoque el cierre inesperado de la aplicación.



Figura D. Esquema del manejo de excepciones y catálogo de errores previstos.

5.5. Pregunta 5

¿Cómo se implementa la persistencia de datos mediante context managers (`with`) y modos de apertura (`r`, `w`, `a`) para que el inventario y el historial de ventas se guarden y recuperen correctamente en cada ejecución?

a) El context manager

La sentencia `with` crea un **context manager**: al entrar en el bloque abre el archivo y, al salir, ejecuta automáticamente su cierre, incluso si dentro del bloque se produce una excepción o se ejecuta un `return`. Esto elimina el error clásico de olvidar `archivo.close()`, que deja el archivo bloqueado para otros programas y puede provocar que la información quede retenida en el búfer sin escribirse en el disco.

```
# Forma aplicada en el sistema
with open(ruta, "w", encoding="utf-8") as f:
    for producto in inventario:
        f.write(producto_a_linea(producto))
# El archivo se cierra aquí automáticamente

# Forma NO utilizada
archivo = open(ruta, "w")
for producto in inventario:
    archivo.write(...)
archivo.close() # si falla antes,
# el archivo queda abierto
```

Comparación entre el uso del context manager y la apertura manual del archivo.

b) Modos de apertura y su justificación

MODO	OPERACIÓN DEL SISTEMA	POR QUÉ ESE MODO
r (lectura)	cargar_inventario() y cargar_ventas(), al iniciar el programa.	Solo se necesita leer. Si el archivo no existe, Python lanza <code>FileNotFoundError</code> , que se captura para arrancar con estructuras vacías en la primera ejecución.
w (escritura)	guardar_inventario(), tras cada alta, actualización, baja o venta.	El catálogo debe quedar exactamente como está en memoria: el modo <code>w</code> reescribe el archivo completo, de modo que las bajas y los cambios de stock desaparecen del registro anterior.
a (agregar)	registrar_venta_en_archivo(), registrar_error() e inicializar_almacenamiento().	El historial de ventas y la bitácora son acumulativos : cada comprobante se añade al final sin alterar los anteriores, lo que preserva la trazabilidad. Además, <code>a</code> crea el archivo si no existe, sin destruir su contenido si ya existe.

c) Ciclo completo de la persistencia

MOMENTO	ACCIÓN	ARCHIVO Y MODO
1. Arranque	<code>inicializar_almacenamiento()</code> verifica o crea la carpeta <code>datos/</code> y los tres archivos.	<code>inventario.txt</code> , <code>ventas.txt</code> , <code>errores.log</code> — modo <code>a</code>
2. Carga	<code>cargar_inventario()</code> y <code>cargar_ventas()</code> leen los archivos y reconstruyen las estructuras en memoria.	modo <code>r + read()</code> / <code>readlines()</code>
3. Operación	El usuario registra productos y ventas; los datos se modifican en memoria.	—
4. Grabado	Tras cada operación relevante se graba el inventario y se agrega la venta.	<code>w</code> para el inventario · <code>a</code> para las ventas
5. Cierre	La opción 0 ejecuta un guardado de seguridad antes de terminar.	modo <code>w</code>

MOMENTO	ACCIÓN	ARCHIVO Y MODO
6. Siguiente ejecución	El sistema vuelve a leer los archivos y recupera el estado exacto en que quedó.	modo r (Figura 15)

d) Formato de almacenamiento

Los datos se guardan en **texto plano delimitado por punto y coma**, formato legible con cualquier editor y compatible con una hoja de cálculo. La conversión entre el texto del archivo y las estructuras de Python se concentra en cuatro funciones de serialización, de modo que un cambio de formato afectaría solo a ellas:

```
# inventario.txt -> id ; nombre ; categoria ; precio ; stock
P001;Audifonos Bluetooth ANC;Audio;159.90;14

# ventas.txt -> boleta ; fecha ; cliente ; items ; subtotal ; igv ; total
#
# donde items = id:cantidad:precio:descuento | id:cantidad:...
B0001;12/09/2026 00:00:06;Maria Quispe Ruiz;P002:6:49.90:8.0|P004:12:35.50:12.0|
P011:1:189.50:0.0;839.83;151.17;991.00
```

Formato de los registros grabados en los archivos de texto plano.

```
def registrar_venta_en_archivo(venta, ruta=ARCHIVO_VENTAS):
    """Agrega UNA venta al final del historial (modo "a" - append)."""
    grabada = False
    try:
        os.makedirs(CARPETA_DATOS, exist_ok=True)
        with open(ruta, "a", encoding="utf-8") as archivo: # modo APPEND
            archivo.write(venta_a_linea(venta))
    except PermissionError as error:
        print(" [!] No se pudo registrar la venta (permisos):", error)
        registrar_error("registrar_venta_en_archivo", str(error))
    except OSError as error:
        print(" [!] Error al grabar la venta:", error)
        registrar_error("registrar_venta_en_archivo", str(error))
    else:
        grabada = True
    finally:
        print(" [i] Operacion de registro de venta finalizada.")
    return grabada
```

Escritura en modo a: cada venta se agrega al final del historial sin sobrescribir las anteriores.

La verificación de la persistencia se documenta en las Figuras 15 y 16: tras cerrar el programa, se vuelve a ejecutar y el sistema informa «Inventario cargado: 11 producto(s)» e «Historial cargado: 1 venta(s)», recuperando el stock ya descontado por la venta registrada en la sesión anterior.

6. CONSIDERACIONES PARA EL ENTREGABLE

Conforme a lo solicitado, este capítulo presenta la **propuesta de solución** para el caso práctico, fundamentada en los conocimientos adquiridos a lo largo del curso, y los **esquemas y diagramas** alineados a dicha propuesta. Se incluyen además los tres componentes exigidos como entregable: el archivo `.py` funcional, el código documentado con capturas de pantalla del menú y de la ejecución, y el manual de explicación de la arquitectura modular.

6.1. Propuesta de solución

6.1.1. Descripción general

Se propone **TecnoStore v1.0**, una aplicación de consola escrita en Python 3 que reemplaza los cuadernos y las hojas de cálculo desconectadas de la microempresa por un único sistema con datos centralizados y persistentes. La aplicación se distribuye como un solo archivo ejecutable, `sistema_inventario.py`, que no requiere instalación de librerías externas ni de un motor de base de datos: basta con tener Python instalado y ejecutar el archivo.

Al iniciarse, el sistema verifica la carpeta de datos, carga el catálogo y el historial de ventas desde los archivos de texto y presenta un menú de diez opciones. Toda operación que modifica la información se graba de inmediato en disco, de modo que un corte de energía o un cierre accidental no ocasiona pérdida de datos.

CARACTERÍSTICA	ESPECIFICACIÓN
Nombre de la solución	TecnoStore v1.0 — Sistema de Gestión de Inventario y Registro de Ventas
Archivo ejecutable	<code>sistema_inventario.py</code> (1 538 líneas, 51 funciones documentadas)
Lenguaje y versión	Python 3.8 o superior (probado en Python 3.11)
Librerías empleadas	Solo la biblioteca estándar: <code>os</code> , <code>sys</code> y <code>datetime</code>
Tipo de interfaz	Consola de texto, operable íntegramente con el teclado
Almacenamiento	Tres archivos de texto plano en la carpeta <code>datos/</code>
Usuarios simultáneos	Uno (operación local en el punto de venta)
Impuesto aplicado	IGV del 18 % calculado sobre el subtotal de cada comprobante
Descuento comercial	Automático por volumen: 5 % desde 3 unidades, 8 % desde 6 y 12 % desde 12

6.1.2. Arquitectura modular

La solución evita por completo la programación lineal o monolítica. El programa se organiza en **cinco capas**, cada una con una responsabilidad definida, y el flujo de llamadas va siempre en un mismo sentido: la interfaz solicita el dato a la capa de validación, entrega el dato validado a la lógica de negocio y esta

devuelve el resultado que la capa de persistencia graba en disco. Ninguna capa inferior llama a una superior.

CAPA	RESPONSABILIDAD	REGLA DE DISEÑO
1. Configuración	Concentra las 9 constantes del sistema: rutas de archivos, IGV, stock mínimo, escala de descuentos y categorías.	Un cambio de política comercial (por ejemplo, un nuevo porcentaje de IGV) se realiza en una sola línea del programa.
2. Persistencia	Único punto del programa que accede al disco. Lee y graba archivos de texto y convierte los registros a estructuras de Python.	Si mañana la empresa migrara a una base de datos, solo se reescribiría esta capa.
3. Validación	Lee del teclado y garantiza que el dato entregado a las capas superiores sea del tipo y del rango correctos.	Ninguna otra capa vuelve a comprobar tipos de dato: la validación ocurre una sola vez y en un solo lugar.
4. Lógica de negocio	Contiene las reglas de la empresa: búsquedas, altas, descuentos, impuestos, control de stock y reportes.	No utiliza <code>print()</code> ni <code>input()</code> ; es reutilizable en una interfaz gráfica o web sin modificación alguna.
5. Interfaz y control	Presenta los menús, las tablas y el comprobante, y coordina el flujo del programa mediante <code>main()</code> .	Concentra el diálogo con el usuario; los cambios de presentación no afectan a las reglas del negocio.

La Figura A (sección 5.3) muestra la distribución completa de las 51 funciones entre las cinco capas.

6.1.3. Inventario de funciones del sistema

Capa 2 — Persistencia de datos (10 funciones)

FUNCIÓN	PARÁMETROS	RETORNO	RESPONSABILIDAD
<code>registrar_error</code>	origen, detalle	<code>bool</code>	Agrega una incidencia a la bitácora <code>errores.log</code> (modo <code>a</code>).
<code>inicializar_almacenamiento</code>	—	<code>bool</code>	Crea la carpeta <code>datos/</code> y los tres archivos si no existen.
<code>linea_a_producto</code>	linea	<code>dict</code>	Convierte un registro de texto en un diccionario producto.
<code>producto_a_linea</code>	producto	<code>str</code>	Convierte un producto en una línea grabable.
<code>cargar_inventario</code>	ruta	<code>list</code>	Lee <code>inventario.txt</code> (modo <code>r</code>) y descarta registros corruptos.
<code>guardar_inventario</code>	inventario, ruta	<code>bool</code>	Reescribe el catálogo completo (modo <code>w</code>).
<code>venta_a_linea</code>	venta	<code>str</code>	Serializa un comprobante con su detalle de ítems.
<code>linea_a_venta</code>	linea	<code>dict</code>	Reconstruye el diccionario anidado de una venta.
<code>cargar_ventas</code>	ruta	<code>list</code>	Lee el historial de ventas (modo <code>r</code>).
<code>registrar_venta_en_archivo</code>	venta, ruta	<code>bool</code>	Agrega un comprobante al historial (modo <code>a</code>).

Capa 3 — Validación de entradas (5 funciones)

FUNCIÓN	PARÁMETROS	RETORNO	RESPONSABILIDAD
leer_cadena	mensaje, obligatorio, maximo, cancelable	str None	Lee texto controlando obligatoriedad, longitud y cancelación con X.
leer_entero	mensaje, minimo, maximo, cancelable	int None	Lee un entero capturando ValueError y verificando el rango.
leer_decimal	mensaje, minimo, maximo, cancelable	float None	Lee un importe; acepta coma o punto como separador decimal.
leer_opcion	mensaje, opciones_validas	str	Lee una opción del menú y verifica que pertenezca a las permitidas.
confirmar	mensaje	bool	Solicita una confirmación S/N antes de una operación irreversible.

Capa 4 — Lógica de negocio (19 funciones)

FUNCIÓN	PARÁMETROS	RETORNO	RESPONSABILIDAD
buscar_producto_por_id	inventario, id_producto	dict None	Localiza un producto por su código único.
existe_id	inventario, id_producto	bool	Verifica duplicados antes de un alta.
buscar_productos_por_texto	inventario, texto	list	Busca coincidencias en nombre y categoría.
generar_id_producto	inventario	str	Propone el siguiente código correlativo (P001, P002, ...).
crear_producto	id, nombre, categoria, precio, stock	dict	Construye y valida el registro de un producto.
agregar_producto	inventario, producto	bool	Da de alta el producto; lanza ValueError si el ID existe.
actualizar_precio	inventario, id, precio	dict	Modifica el precio de venta de un producto.
actualizar_stock	inventario, id, movimiento	int	Suma o resta unidades controlando que el stock no sea negativo.
eliminar_producto	inventario, id_producto	dict	Da de baja un producto del catálogo.
calcular_descuento	cantidad	float	Determina el porcentaje de descuento por volumen.
calcular_importe_item	cantidad, precio, descuento	float	Calcula el importe neto de una línea de venta.
crear_item_venta	producto, cantidad	dict	Arma el detalle validando la disponibilidad de stock.
generar_numero_boleta	ventas	str	Genera el correlativo del comprobante (B0001, B0002, ...).
construir_venta	numero, cliente, items	dict	Consolida el comprobante con subtotal, IGV y total.
descontar_stock_de_venta	inventario, items	int	Descuenta del inventario las unidades vendidas.
valorizar_inventario	inventario	float	Calcula el valor del inventario a precio de venta.
productos_stock_critico	inventario, minimo	list	Identifica los productos que requieren reposición.
resumen_ventas	ventas, fecha	dict	Consolida comprobantes, unidades, IGV, total y ticket promedio.
ranking_productos	ventas, top	list	Determina los productos más vendidos del historial.

Capa 5 — Interfaz de consola y control (17 funciones)

FUNCIÓN	PARÁMETROS	RESPONSABILIDAD
limpiar_pantalla	–	Limpia la consola según el sistema operativo.
mostrar_encabezado	titulo	Imprime el encabezado estándar de cada módulo.
mostrar_menu_principal	–	Presenta las diez opciones del sistema.
mostrar_tabla_productos	productos	Muestra los productos en tabla alineada con su estado de stock.
mostrar_comprobante	venta	Imprime la boleta de venta con detalle, IGV y total.
pausar	–	Detiene la ejecución hasta que el usuario presione ENTER.
opcion_listar_inventario	inventario	Opción 1: listado general y valorizado del catálogo.
opcion_registrar_producto	inventario	Opción 2: alta de producto con control de duplicados.
opcion_buscar_producto	inventario	Opción 3: búsqueda por código o por texto libre.
opcion_actualizar_producto	inventario	Opción 4: cambio de precio, ingreso, merma o baja.
opcion_registrar_venta	inventario, ventas	Opción 5: registro de la venta y emisión del comprobante.
opcion_historial_ventas	ventas	Opción 6: listado de comprobantes emitidos.
opcion_reporte_inventario	inventario	Opción 7: reporte por categoría y alerta de stock crítico.
opcion_reporte_ventas	ventas	Opción 8: ventas del día, acumulado y ranking de productos.
opcion_guardar_cambios	inventario	Opción 9: grabado manual del inventario en disco.
cargar_datos_iniciales	–	Prepara el almacenamiento y carga los datos al arrancar.
main	–	Controlador del menú principal (while + if-elif-else).

6.1.4. Reglas de negocio implementadas

REGLA	DEFINICIÓN	FUNCIÓN RESPONSABLE
Código único de producto	No se admiten códigos duplicados; el sistema propone el siguiente correlativo disponible.	existe_id() · generar_id_producto()
Stock no negativo	Ninguna operación puede dejar el stock por debajo de cero; la venta se rechaza si la cantidad supera lo disponible.	actualizar_stock() · crear_item_venta()
Descuento por volumen	5 % desde 3 unidades, 8 % desde 6 unidades y 12 % desde 12 unidades del mismo producto.	calcular_descuento()
Impuesto general a las ventas	IGV del 18 % aplicado sobre el subtotal del comprobante, con redondeo a dos decimales.	construir_venta()
Alerta de stock crítico	Todo producto con 5 unidades o menos se marca como CRÍTICO; con 0 unidades, como AGOTADO.	productos_stock_critico()
Correlatividad de comprobantes	Los comprobantes se numeran de forma consecutiva a partir del historial existente (B0001, B0002, ...).	generar_numero_boleta()
Persistencia inmediata	Toda operación que modifica datos graba el archivo antes de volver al menú.	guardar_inventario() · registrar_venta_en_archivo())

6.1.5. Estructura de archivos del proyecto

```
trabajo-final-piad221/
├── codigo/
│   ├── sistema_inventario.py    <- ENTREGABLE: aplicacion de consola
│   ├── datos/                  <- se genera en la primera ejecucion
│   └── inventario.txt           <- catalogo de productos (modo w)
```

└─ ventas.txt	<- historial de ventas (modo a)
└─ errores.log	<- bitacora de incidencias (modo a)
└─ diagramas/	<- Figuras A, B, C, D y E
└─ capturas/	<- 16 capturas de la ejecucion real
└─ documento/	
└─ PIAD-221_Trabajo_Final.docx	<- el presente informe

Organización de las carpetas del trabajo entregado.

6.2. Esquemas y diagramas de la propuesta

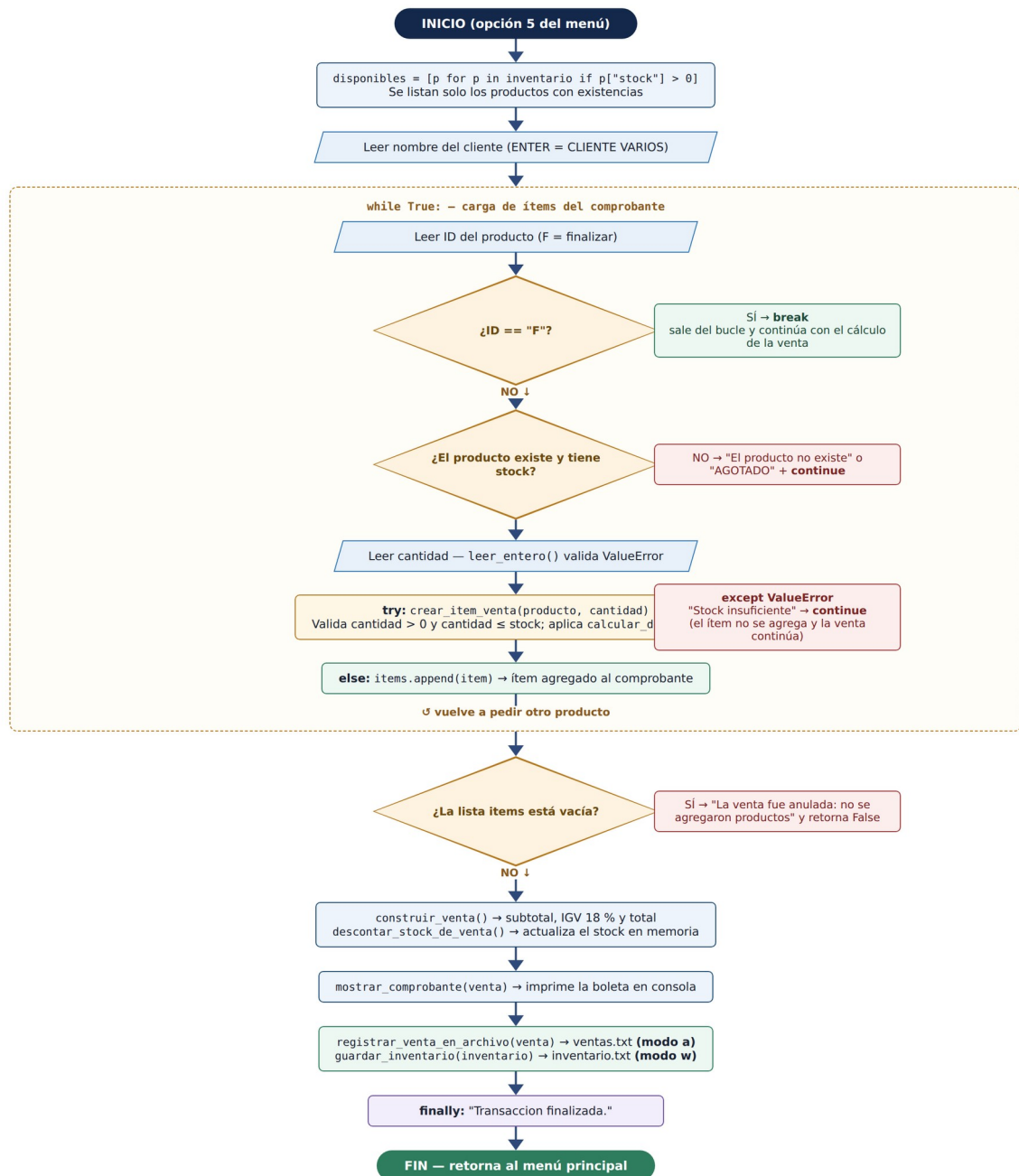
Se elaboraron cinco esquemas alineados a la propuesta de solución. Tres de ellos ya fueron presentados junto con las preguntas guía que fundamentan; en este apartado se listan todos y se incluye el diagrama del proceso central del sistema: el registro de una venta.

FIGURA	DENOMINACIÓN	QUÉ REPRESENTA	UBICACIÓN
A	Arquitectura modular	Las cinco capas del sistema y la distribución de las 51 funciones entre ellas.	Sección 5.3
B	Diagrama de flujo del programa principal	El bucle <code>while</code> del menú, la derivación <code>if-elif-else</code> y la salida controlada con <code>break</code> .	Sección 5.2
C	Modelo de datos y persistencia	Las estructuras en memoria (lista de diccionarios y diccionario anidado) y su correspondencia con los archivos de texto.	Sección 5.1
D	Esquema del manejo de excepciones	La semántica de <code>try-except-else-finally</code> y el catálogo de errores previstos.	Sección 5.4
E	Diagrama de flujo del registro de venta	La transacción completa: carga de ítems, validaciones, cálculo de descuentos e IGV y persistencia.	Esta sección

El registro de venta es la operación más compleja del sistema porque combina un bucle de carga de ítems, tres validaciones sucesivas, el cálculo de descuentos e impuestos, la actualización del inventario y dos escrituras en disco con modos distintos. La Figura E detalla ese flujo:

Figura E. Diagrama de flujo del proceso de registro de venta — `opcion_registrar_venta()`

Transacción completa: carga de ítems con validación de stock, cálculo de descuentos e IGV, actualización del inventario y persistencia del comprobante.



La transacción es **segura**: si un ítem falla la validación, solo se descarta ese ítem (continue); si la venta no llega a construirse, el stock no se modifica ni se graba nada en disco.

Figura E. Diagrama de flujo del proceso de registro de venta.

6.3. Manual de explicación de la arquitectura modular

Este manual breve permite a un tercero comprender, ejecutar y mantener la aplicación sin necesidad de leer todo el código fuente.

6.3.1. Requisitos e instalación

REQUISITO	DETALLE
Sistema operativo	Windows 10/11, Linux o macOS
Intérprete	Python 3.8 o superior (descargable desde python.org)
Librerías externas	Ninguna: el programa emplea solo la biblioteca estándar
Espacio en disco	Menos de 1 MB, incluidos los archivos de datos
Permisos	Permiso de escritura en la carpeta donde reside el archivo .py

6.3.2. Puesta en marcha

- 1) Copiar el archivo sistema_inventario.py en una carpeta con permisos de escritura.
- 2) Abrir la consola (CMD, PowerShell o Terminal) en esa carpeta.
- 3) Ejecutar: `python sistema_inventario.py`
(en Linux/macOS: `python3 sistema_inventario.py`)
- 4) El sistema crea automáticamente la carpeta datos/ con los tres archivos.
- 5) Presionar ENTER para acceder al menú principal.

Secuencia de puesta en marcha del sistema (ver Figura 1).

6.3.3. Guía de uso del menú

OPCIÓN	FUNCIÓN QUE SE EJECUTA	QUÉ HACE EL SISTEMA	EVIDENCIA
1	<code>opcion_listar_inventario()</code>	Muestra el catálogo completo con precio, stock, estado y valorizado total.	Figura 3
2	<code>opcion_registrar_producto()</code>	Solicita los datos del nuevo producto, valida el código duplicado y graba el catálogo.	Figuras 4 y 5
3	<code>opcion_buscar_producto()</code>	Busca por código exacto o por texto libre en nombre y categoría.	Figura 6
4	<code>opcion_actualizar_producto()</code>	Permite cambiar el precio, registrar un ingreso de mercadería, registrar una merma o dar de baja el producto.	Figura 7
5	<code>opcion_registrar_venta()</code>	Registra la venta de varios productos, aplica descuentos e IGV, emite el comprobante y descuenta el stock.	Figuras 8 y 9
6	<code>opcion_historial_ventas()</code>	Lista los comprobantes emitidos con su total y el acumulado recaudado.	Figura 10
7	<code>opcion_reporte_inventario()</code>	Presenta el inventario valorizado, el resumen por categoría y la alerta de reposición.	Figura 11
8	<code>opcion_reporte_ventas()</code>	Muestra las ventas del día, el acumulado histórico, el ticket promedio y el ranking de productos.	Figura 12
9	<code>opcion_guardar_cambios()</code>	Fuerza el grabado del inventario en el archivo de texto.	Figura 13
0	Salida controlada en <code>main()</code>	Graba el inventario, muestra el resumen de la sesión y finaliza con <code>break</code> .	Figura 14

6.3.4. Convenciones de la interfaz

SÍMBOLO / TECLA	SIGNIFICADO
[OK]	La operación se completó correctamente.
[!]	Advertencia o dato inválido: el sistema vuelve a solicitar la información.
[i]	Mensaje informativo del bloque <code>finally</code> : la operación terminó (con éxito o no).
Tecla X	Cancela la operación en curso y regresa al menú principal.
Tecla F	Finaliza la carga de productos durante el registro de una venta.
ENTER	Acepta el valor propuesto entre corchetes o continúa al menú.
Ctrl + C	Interrumpe la digitación; el sistema realiza un cierre seguro sin perder datos.

6.3.5. Cómo modificar el sistema

La arquitectura modular permite realizar los cambios más habituales sin tocar el resto del programa:

SI SE DESEA...	MODIFICAR	CAPA AFECTADA
Cambiar el porcentaje de IGV	La constante IGV (una sola línea).	Capa 1
Modificar la escala de descuentos	La constante <code>ESCALA_DESCUENTOS</code> .	Capa 1
Agregar una categoría de productos	La lista <code>CATEGORIAS</code> .	Capa 1
Cambiar el umbral de stock crítico	La constante <code>STOCK_MINIMO</code> .	Capa 1
Cambiar el formato del archivo de datos	Las funciones <code>producto_a_linea()</code> y <code>linea_a_producto()</code> .	Capa 2
Agregar una nueva opción al menú	Una función <code>opcion_...</code> () nueva, la constante <code>OPCIONES_MENU</code> y una rama <code>elif</code> en <code>main()</code> .	Capas 1 y 5
Migrar a una interfaz gráfica	Solo las capas 3 y 5; la lógica de negocio se reutiliza íntegramente.	Capas 3 y 5

6.3.6. Recuperación ante incidencias

SITUACIÓN	COMPORTAMIENTO DEL SISTEMA	ACCIÓN DEL USUARIO
Se digita texto donde se espera un número	Muestra el aviso y vuelve a solicitar el dato; no se pierde lo ya ingresado.	Volver a escribir el valor correcto.
Se intenta vender más unidades que las disponibles	Rechaza el ítem con el mensaje "Stock insuficiente" y continúa la venta.	Corregir la cantidad o elegir otro producto.
No existen los archivos de datos	Informa la ausencia y arranca con el inventario vacío.	Registrar los productos con la opción 2.
El archivo de datos está abierto en otro programa	Informa el error de permisos, lo registra y mantiene el menú operativo.	Cerrar el archivo y usar la opción 9 para grabar.
Una línea del archivo está dañada	Descarta solo esa línea, la registra en la bitácora y carga el resto.	Revisar <code>datos/errores.log</code> y corregir el registro.
Se presiona Ctrl+C	Cancela la operación y ejecuta un cierre seguro.	Volver a ejecutar el programa: los datos están grabados.

6.4. Código fuente documentado

El entregable principal es el archivo `sistema_inventario.py`. Su documentación sigue tres niveles, conforme a las buenas prácticas de la industria y a la guía de estilo PEP 8:

- **Cabecera del archivo:** identifica el trabajo, describe la solución, resume la arquitectura de capas, enumera los archivos de datos generados e indica la forma de ejecución.
- **Comentarios de bloque:** separan visualmente las cinco capas y explican el propósito y las reglas de diseño de cada una.
- **Docstring por función:** las 51 funciones documentan su propósito, sus parámetros, su valor de retorno y las excepciones que pueden lanzar. Además, se añaden comentarios de línea en las instrucciones con mayor carga técnica.

A continuación se presentan los fragmentos más representativos, tal como aparecen en el archivo entregado. El código completo se incluye en el **Anexo A**.

6.4.1. Cabecera y documentación del archivo

```
# -*- coding: utf-8 -*-
"""
=====
SISTEMA DE GESTION DE INVENTARIO Y REGISTRO DE VENTAS PARA UNA MICROEMPRESA
Aplicacion de consola desarrollada en Python 3
=====
Curso      : PIAD-221 - Algoritmos y Programacion para Desarrollo de Software
Carrera    : Tecnologias de la Informacion
Trabajo    : Trabajo Final de Curso
Caso       : Tienda de accesorios tecnologicos "TecnoStore"
Estudiante : [COLOCAR NOMBRES Y APELLIDOS]
ID         : [COLOCAR ID / CODIGO DE MATRICULA]
Instructor : [COLOCAR NOMBRE DEL INSTRUCTOR]
Fecha      : [COLOCAR FECHA DE ENTREGA]
=====
DESCRIPCION GENERAL
=====
Aplicacion de consola que automatiza el registro, consulta y actualizacion de
productos del inventario, asi como el procesamiento de transacciones de venta
de una microempresa dedicada a la comercializacion de accesorios tecnologicos.

La solucion sustituye el registro manual en cuadernos y hojas de calculo
desconectadas, eliminando la desactualizacion del stock, los errores humanos
de calculo y la perdida de tiempo en la busqueda de productos.
```

Líneas 1 a 24 de `sistema_inventario.py`: identificación del trabajo y descripción de la solución.

6.4.2. Capa 1 — Constantes de configuración

```
NOMBRE_EMPRESA = "TECNOSTORE E.I.R.L."
RUBRO_EMPRESA = "Venta de accesorios tecnologicos"

CARPETA_DATOS = "datos" # Carpeta de persistencia
ARCHIVO_INVENTARIO = os.path.join(CARPETA_DATOS, "inventario.txt")
ARCHIVO_VENTAS = os.path.join(CARPETA_DATOS, "ventas.txt")
ARCHIVO_LOG = os.path.join(CARPETA_DATOS, "errores.log")

SEPARADOR_CAMPO = ":" # Separa los campos de un registro
SEPARADOR_ITEM = "|" # Separa los items dentro de una venta
SEPARADOR_SUBCAMPO = ":" # Separa los datos dentro de un item

IGV = 0.18 # Impuesto General a las Ventas (18%)
```



```

STOCK_MINIMO = 5          # Umbral de alerta de stock critico
ANCHO = 78                # Ancho estandar de la consola

# Reglas de descuento por volumen: (cantidad minima, porcentaje de descuento)
ESCALA_DESCUENTOS = [(12, 12.0), (6, 8.0), (3, 5.0)]

CATEGORIAS = ["Audio", "Computo", "Conectividad", "Energia", "Accesorios"]

# Opciones validas del menu principal (se usan para validar la navegacion)
OPCIONES_MENU = ["1", "2", "3", "4", "5", "6", "7", "8", "9", "0"]

```

Las constantes centralizan las políticas del negocio: un cambio de IGV o de escala de descuentos se realiza en una sola línea.

6.4.3. Capa 2 — Persistencia con context manager

La función de carga ilustra la estrategia de robustez adoptada: un registro dañado no interrumpe la lectura, se descarta con `continue` y se deja constancia en la bitácora.

```

def cargar_inventario(ruta=ARCHIVO_INVENTARIO):
    """Carga el catalogo de productos desde el archivo de texto (modo "r").

    Los registros corruptos NO detienen la carga: se informan, se registran en
    la bitacora y el proceso continua con el resto de lineas (continue).

    Parametros:
        ruta (str): ubicacion del archivo de inventario.
    Retorna:
        list: lista de diccionarios con los productos cargados.
    """
    inventario = []
    try:
        with open(ruta, "r", encoding="utf-8") as archivo:
            contenido = archivo.read()          # Metodo read()
    except FileNotFoundError:
        print(" [!] No se encontro '{0}'. Se iniciara un inventario vacio.".format(ruta))
        registrar_error("cargar_inventario", "FileNotFoundError: " + ruta)
    except PermissionError as error:
        print(" [!] Archivo de inventario bloqueado por el sistema:", error)
        registrar_error("cargar_inventario", str(error))
    except UnicodeDecodeError as error:
        print(" [!] El archivo de inventario tiene una codificacion invalida.")
        registrar_error("cargar_inventario", str(error))
    else:
        numero = 0
        for linea in contenido.splitlines():
            numero += 1
            if not linea.strip():
                continue                        # Ignora lineas en blanco
            try:
                producto = linea_a_producto(linea)
            except ValueError as error:
                print(" [!] Linea {0} descartada del inventario ({1}).".format(numero,
error))
                registrar_error("cargar_inventario", "Linea {0}: {1}".format(numero,
error))
                continue                        # El programa NO se interrumpe
            inventario.append(producto)
            print(" [OK] Inventario cargado: {0} producto(s)".format(len(inventario)))
        finally:
            print(" [i] Proceso de lectura del inventario finalizado.")
    return inventario

```

Lectura del catálogo en modo r con try-except-else-finally y tolerancia a registros corruptos.

6.4.4. Capa 3 — Validación de entradas

```

def leer_entero(mensaje, minimo=None, maximo=None, cancelable=True):
    """Lee un numero entero validando tipo de dato y rango permitido.

    Evita el error tipico de convertir texto no numerico con int(), capturando
    la excepcion ValueError y solicitando nuevamente el dato.

    Parametros:

```

```

    mensaje (str): indicacion mostrada al usuario.
    minimo (int): valor minimo aceptado (None = sin limite inferior).
    maximo (int): valor maximo aceptado (None = sin limite superior).
    cancelable(bool): si True, la letra X cancela la operacion.
Retorna:
    int : numero entero validado, o None si el usuario cancela.
"""
while True:
    entrada = ""
    try:
        entrada = input(mensaje).strip()
        if cancelable and entrada.upper() == "X":
            return None
        numero = int(entrada) # Puede lanzar ValueError
    except ValueError:
        print(" [!] Dato invalido: '{0}' no es un numero entero.".format(entrada))
        registrar_error("leer_entero", "ValueError con la entrada: " + entrada)
        continue
    except (EOFError, KeyboardInterrupt):
        print("\n [i] Entrada interrumpida por el usuario.")
        return None
    else:
        # El bloque else solo se ejecuta si NO hubo excepcion.
        if minimo is not None and numero < minimo:
            print(" [!] El valor minimo permitido es {0}.".format(minimo))
            continue
        if maximo is not None and numero > maximo:
            print(" [!] El valor maximo permitido es {0}.".format(maximo))
            continue
        return numero

```

Validación de un número entero: se captura ValueError, se verifica el rango en el bloque else y se reintenta con continue.

6.4.5. Capa 4 — Lógica de negocio

Las funciones de negocio no dialogan con el usuario: reciben datos, aplican la regla y devuelven el resultado o lanzan una excepción. Este es el cálculo comercial del sistema:

```

def calcular_descuento(cantidad):
    """Determina el porcentaje de descuento por volumen de compra.

    Escala comercial vigente:
        12 unidades o mas ... 12 %
        6 a 11 unidades .... 8 %
        3 a 5 unidades .... 5 %
        menos de 3 ..... sin descuento

    Parametros:
        cantidad (int): unidades solicitadas del producto.
    Retorna:
        float: porcentaje de descuento aplicable.
    """
    for minimo, porcentaje in ESCALA_DESCUENTOS:
        if cantidad >= minimo:
            return porcentaje
    return 0.0

def crear_item_venta(producto, cantidad):
    """Construye la linea de detalle de una venta a partir de un producto.

    Parametros:
        producto (dict): producto del inventario.
        cantidad (int) : unidades solicitadas.
    Retorna:
        dict: item con cantidad, precio, descuento e importe.
    Lanza:
        ValueError: si la cantidad no es positiva o supera el stock.
    """
    if cantidad <= 0:
        raise ValueError("La cantidad debe ser mayor que cero")
    if cantidad > producto["stock"]:
        raise ValueError("Stock insuficiente: solo quedan {0} unidad(es)".format(
            producto["stock"]))
    descuento = calcular_descuento(cantidad)

```

```

return {
    "id": producto["id"],
    "nombre": producto["nombre"],
    "cantidad": cantidad,
    "precio_unitario": producto["precio"],
    "descuento": descuento,
    "importe": calcular_importe_item(cantidad, producto["precio"], descuento),
}

```

Escala de descuentos por volumen y construcción de una línea de venta con control de stock.

```

def construir_venta(numero_boleta, cliente, items):
    """Arma el comprobante de venta con sus totales e impuestos.

    Estructura resultante (diccionario anidado):
        {boleta, fecha, cliente, items:[...], {...]}, subtotal, igv, total}

    Parametros:
        numero_boleta (str) : correlativo del comprobante.
        cliente (str) : nombre del cliente.
        items (list): lineas de detalle de la venta.

    Retorna:
        dict: comprobante de venta completo.

    Lanza:
        ValueError: si la venta no contiene items.
    """
    if not items:
        raise ValueError("La venta debe contener al menos un producto")
    subtotal = round(sum(item["importe"] for item in items), 2)
    igv = round(subtotal * IGv, 2)
    return {
        "boleta": numero_boleta,
        "fecha": datetime.now().strftime("%d/%m/%Y %H:%M:%S"),
        "cliente": cliente if cliente else "CLIENTE VARIOS",
        "items": items,
        "subtotal": subtotal,
        "igv": igv,
        "total": round(subtotal + igv, 2),
    }

```

Consolidación del comprobante: subtotal, IGv del 18 % y total, en una estructura de diccionario anidado.

6.4.6. Capa 5 — Controlador del menú principal

```

def main():
    """Punto de entrada del sistema: menu principal persistente.

    Estructura de control:
        while True ..... mantiene el menu activo hasta elegir salir.
        if - elif - else ..... deriva la opcion hacia su funcion modular.
        break ..... unica salida controlada del bucle.
    """
    inventario, ventas = cargar_datos_iniciales()
    pausar()

    while True:
        limpiar_pantalla()
        mostrar_menu_principal()
        opcion = leer_opcion(" Seleccione una opcion [0-9]: ", OPCIONES_MENU)
        print()

        if opcion == "1":
            opcion_listar_inventario(inventario)
        elif opcion == "2":
            opcion_registrar_producto(inventario)
        elif opcion == "3":
            opcion_buscar_producto(inventario)
        elif opcion == "4":
            opcion_actualizar_producto(inventario)
        elif opcion == "5":
            opcion_registrar_venta(inventario, ventas)
        elif opcion == "6":
            opcion_historial_ventas(ventas)
        elif opcion == "7":
            opcion_reporte_inventario(inventario)
        elif opcion == "8":
            opcion_reporte_ventas(ventas)

```

```

elif opcion == "9":
    opcion_guardar_cambios(inventario)
else:
    # Opcion "0": salida controlada con guardado de seguridad.
    mostrar_encabezado("Cierre del sistema")
    guardar_inventario(inventario)
    print("\n Resumen de la sesion:")
    print("  - Productos en catalogo : {0}".format(len(inventario)))
    print("  - Ventas registradas      : {0}".format(len(ventas)))
    print("  - Valorizado final          : S/ {0:,.2f}".format(
        valorizar_inventario(inventario)))
    print("\n Gracias por utilizar el sistema. Hasta pronto!")
    print("=" * ANCHO)
    break # Unica salida del bucle principal

pausar()

```

Función main(): bucle while persistente, derivación if-elif-else y salida controlada con break.

6.4.7. Arranque protegido del programa

```

# =====
# ARRANQUE DEL PROGRAMA
# -----
# La condicion __name__ == "__main__" permite que el archivo se ejecute como
# programa y, a la vez, que sus funciones puedan importarse desde otro modulo
# sin que el menu se dispare automaticamente.
# =====

if __name__ == "__main__":
    try:
        main()
    except KeyboardInterrupt:
        print("\n\n [i] Ejecucion interrumpida con Ctrl+C. Cierre seguro del sistema.")
    except Exception as error:
        # Red de seguridad de ultimo nivel
        print("\n [!] Error inesperado: {0}".format(error))
        registrar_error("main", "{0}: {1}".format(type(error).__name__, error))
        print(" [i] El detalle tecnico se registro en '{0}'".format(ARCHIVO_LOG))
    finally:
        print(" [i] Proceso finalizado.Codigo de salida 0.")
        sys.exit(0)

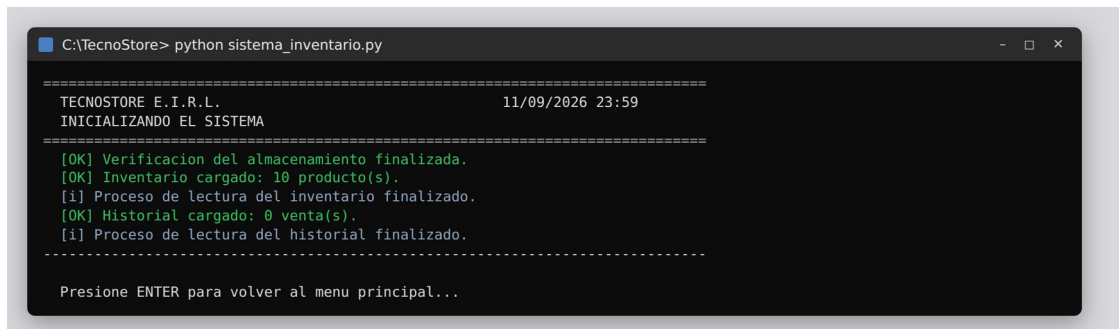
```

*Red de seguridad final: ninguna excepción imprevista muestra un *traceback* al usuario.*

6.5. Capturas de pantalla del menú y de la ejecución

Las dieciséis capturas siguientes corresponden a **una sesión real y continua** de la aplicación, ejecutada en consola. Se documenta el menú principal, cada una de las operaciones y, de forma deliberada, el comportamiento del sistema ante entradas incorrectas.

Al ejecutarse, el sistema comprueba la carpeta `datos/`, lee `inventario.txt` y `ventas.txt` e informa cuántos registros recuperó. Los mensajes `[i]` provienen de los bloques `finally`, que se ejecutan siempre.



```
C:\TecnoStore> python sistema_inventario.py

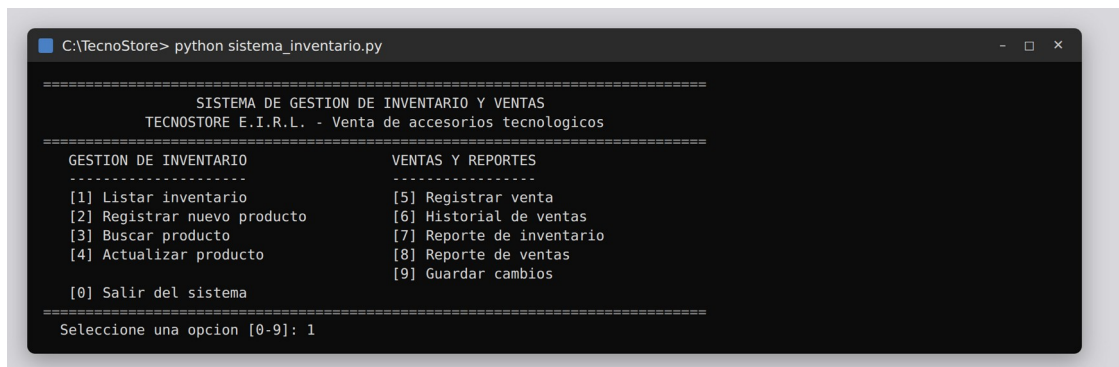
=====
TECNOSTORE E.I.R.L.                                11/09/2026 23:59
INICIALIZANDO EL SISTEMA
=====

[OK] Verificación del almacenamiento finalizada.
[OK] Inventario cargado: 10 producto(s).
[i] Proceso de lectura del inventario finalizado.
[OK] Historial cargado: 0 venta(s).
[i] Proceso de lectura del historial finalizado.
=====

Presione ENTER para volver al menu principal...
```

Figura 1. Inicio del sistema: verificación del almacenamiento y carga de los archivos de datos.

El menú se genera con la función `mostrar_menu_principal()` y permanece activo gracias al bucle `while True`. La opción digitada se valida contra la lista `OPCIONES_MENU` antes de derivarse.



```
C:\TecnoStore> python sistema_inventario.py

=====
                SISTEMA DE GESTION DE INVENTARIO Y VENTAS
                TECNOSTORE E.I.R.L. - Venta de accesorios tecnologicos
=====

GESTION DE INVENTARIO                                VENTAS Y REPORTES
-----
[1] Listar inventario                                [5] Registrar venta
[2] Registrar nuevo producto                         [6] Historial de ventas
[3] Buscar producto                                 [7] Reporte de inventario
[4] Actualizar producto                             [8] Reporte de ventas
[0] Salir del sistema                               [9] Guardar cambios
=====

Seleccione una opcion [0-9]: 1
```

Figura 2. Menú principal interactivo con las diez opciones del sistema.

Cada producto muestra su estado calculado: OK, CRÍTICO (5 unidades o menos) o AGOTADO. Al pie se presenta el valorizado del inventario a precio de venta.

```
C:\TecnoStore> python sistema_inventario.py

=====
SISTEMA DE GESTION DE INVENTARIO Y VENTAS
TECNOSTORE E.I.R.L. - Venta de accesorios tecnologicos
=====

GESTION DE INVENTARIO          VENTAS Y REPORTES
-----
[1] Listar inventario          [5] Registrar venta
[2] Registrar nuevo producto   [6] Historial de ventas
[3] Buscar producto            [7] Reporte de inventario
[4] Actualizar producto        [8] Reporte de ventas
                                [9] Guardar cambios

[0] Salir del sistema

=====
Seleccione una opcion [0-9]: 1

=====
TECNOSTORE E.I.R.L.                  11/09/2026 23:59
LISTADO GENERAL DEL INVENTARIO
=====

ID      PRODUCTO                  CATEGORIA      PRECIO S/   STOCK   ESTADO
-----
P001    Audifonos Bluetooth ANC      Audio           159.90      14      OK
P002    Mouse Inalambrico 2.4G         Computo         49.90      22      OK
P003    Teclado Mecanico RGB           Computo        229.00       8      OK
P004    Cable HDMI 2.1 - 2m           Conectividad    35.50      40      OK
P005    Power Bank 20000 mAh           Energia        129.00       4      CRITICO
P006    Memoria USB 128 GB            Computo         69.90      17      OK
P007    Hub USB-C 7 en 1              Conectividad    119.00       6      OK
P008    Cargador GaN 65W              Energia        149.00       2      CRITICO
P009    Parlante Bluetooth 20W         Audio           189.00       9      OK
P010    Soporte Laptop Aluminio        Accesorios      89.90      12      OK

=====
Total de productos listados: 10
Valorizado total del inventario: S/ 12,084.50

Presione ENTER para volver al menu principal...
```

Figura 3. Opción 1: listado general del inventario con valorizado total.

Se digitó "ciento ochenta" en el precio y "-8" en el stock. El sistema capturó la `ValueError`, rechazó el valor negativo por rango y volvió a solicitar cada dato sin cerrarse. El código correlativo P011 fue propuesto automáticamente.

```
C:\TecnoStore> python sistema_inventario.py

=====
SISTEMA DE GESTION DE INVENTARIO Y VENTAS
TECNOSTORE E.I.R.L. - Venta de accesorios tecnologicos
=====

GESTION DE INVENTARIO          VENTAS Y REPORTES
-----
[1] Listar inventario          [5] Registrar venta
[2] Registrar nuevo producto   [6] Historial de ventas
[3] Buscar producto           [7] Reporte de inventario
[4] Actualizar producto       [8] Reporte de ventas
[0] Salir del sistema         [9] Guardar cambios

=====
Seleccione una opcion [0-9]: 2

=====
TECNOSTORE E.I.R.L.                  11/09/2026 23:59
REGISTRO DE NUEVO PRODUCTO
=====

(Escriba X en cualquier momento para cancelar la operacion)

ID del producto [ENTER = P011]:
Nombre del producto      : Webcam Full HD 1080p
Categorias disponibles: 1=Audio, 2=Computo, 3=Conectividad, 4=Energia, 5=Accesorios
Categoria (1-5)         : 2
Precio unitario de venta S/ : ciento ochenta
[!] Dato invalido: ingrese un monto numerico (ej. 59.90).
Precio unitario de venta S/ : 189.50
Stock inicial (unidades)  : -8
[!] El valor minimo permitido es 0.
Stock inicial (unidades)  : 15

[OK] Producto 'Webcam Full HD 1080p' registrado con el codigo P011.
[OK] Inventario grabado en 'datos/inventario.txt' (11 registro/s).
[i] Operacion de guardado finalizada.
[i] Fin del proceso de alta de producto.

Presione ENTER para volver al menu principal...
```

Figura 4. Opción 2: alta de producto con validación de entradas incorrectas.

Al intentar registrar el código P003, que ya existe en el catálogo, la función `existe_id()` bloquea el alta y devuelve el control al menú principal.

```
C:\TecnoStore> python sistema_inventario.py

=====
SISTEMA DE GESTION DE INVENTARIO Y VENTAS
TECNOSTORE E.I.R.L. - Venta de accesorios tecnologicos
=====

GESTION DE INVENTARIO          VENTAS Y REPORTES
-----
[1] Listar inventario          [5] Registrar venta
[2] Registrar nuevo producto   [6] Historial de ventas
[3] Buscar producto           [7] Reporte de inventario
[4] Actualizar producto       [8] Reporte de ventas
[0] Salir del sistema         [9] Guardar cambios

=====
Seleccione una opcion [0-9]: 2

=====
TECNOSTORE E.I.R.L.                  11/09/2026 23:59
REGISTRO DE NUEVO PRODUCTO
=====

(Escriba X en cualquier momento para cancelar la operacion)

ID del producto [ENTER = P012]: P003
[!] El ID 'P003' ya existe. No se permiten duplicados.

Presione ENTER para volver al menu principal...
```

Figura 5. Opción 2: control de identificadores duplicados.

La búsqueda por el texto "Computo" devolvió los cuatro productos de esa categoría, resolviendo el problema de la pérdida de tiempo en la localización de artículos.

```
C:\TecnoStore> python sistema_inventario.py

=====
SISTEMA DE GESTION DE INVENTARIO Y VENTAS
TECNOSTORE E.I.R.L. - Venta de accesorios tecnologicos
=====

GESTION DE INVENTARIO          VENTAS Y REPORTES
-----
[1] Listar inventario          [5] Registrar venta
[2] Registrar nuevo producto   [6] Historial de ventas
[3] Buscar producto            [7] Reporte de inventario
[4] Actualizar producto        [8] Reporte de ventas
[0] Salir del sistema          [9] Guardar cambios
=====

Seleccione una opcion [0-9]: 3

=====
TECNOSTORE E.I.R.L.          12/09/2026 00:00
BUSQUEDA DE PRODUCTOS
=====

[1] Buscar por ID exacto
[2] Buscar por nombre o categoria

Seleccione el tipo de busqueda: 2
Ingrese el texto a buscar: Computo

=====
ID      PRODUCTO                CATEGORIA    PRECIO S/  STOCK  ESTADO
=====
P002    Mouse Inalambrico 2.4G        Computo      49.90      22     OK
P003    Teclado Mecanico RGB          Computo      229.00     8      OK
P006    Memoria USB 128 GB            Computo      69.90     17     OK
P011    Webcam Full HD 1080p          Computo     189.50     15     OK
=====

Total de productos listados: 4

Presione ENTER para volver al menu principal...
```

Figura 6. Opción 3: búsqueda de productos por nombre o categoría.

El producto P008 se encontraba en estado CRÍTICO con 2 unidades. Tras registrar el ingreso de 20 unidades, el stock quedó en 22 y el inventario se grabó de inmediato en disco.

```
C:\Tecnostore> python sistema_inventario.py

=====
SISTEMA DE GESTION DE INVENTARIO Y VENTAS
TECNOSTORE E.I.R.L. - Venta de accesorios tecnologicos
=====

GESTION DE INVENTARIO          VENTAS Y REPORTES
-----
[1] Listar inventario          [5] Registrar venta
[2] Registrar nuevo producto   [6] Historial de ventas
[3] Buscar producto           [7] Reporte de inventario
[4] Actualizar producto        [8] Reporte de ventas
[0] Salir del sistema          [9] Guardar cambios
=====

Seleccione una opcion [0-9]: 4

=====
TECNOSTORE E.I.R.L.                  12/09/2026 00:00
ACTUALIZACION DE PRODUCTOS
=====

ID del producto a actualizar (X = cancelar): P008

=====
ID      PRODUCTO          CATEGORIA      PRECIO S/  STOCK  ESTADO
=====
P008    Cargador GaN 65W   Energia        149.00     2    CRITICO
=====

Total de productos listados: 1

[1] Modificar precio de venta
[2] Registrar ingreso de mercaderia (sumar stock)
[3] Registrar merma o ajuste (restar stock)
[4] Dar de baja el producto

Seleccione la accion: 2
Unidades que ingresan: 20

[OK] Stock actualizado. Nuevo stock: 22 unidad(es).
[OK] Inventario grabado en 'datos/inventario.txt' (11 registro/s).
[i] Operacion de guardado finalizada.
[i] Fin del proceso de actualizacion.

Presione ENTER para volver al menu principal...
```

Figura 7. Opción 4: registro de un ingreso de mercadería.

La captura documenta tres situaciones: un código inexistente (P099), una cantidad no numérica ("seis") y una cantidad superior al stock disponible (80 unidades de un producto con 40). En los tres casos el sistema avisó y continuó la operación mediante `continue`; además, se aplicaron automáticamente los descuentos del 8 % y del 12 % por volumen.

```
C:\TecnoStore> python sistema_inventario.py

=====
SISTEMA DE GESTION DE INVENTARIO Y VENTAS
TECNOSTORE E.I.R.L. - Venta de accesorios tecnologicos
=====

GESTION DE INVENTARIO          VENTAS Y REPORTES
-----
[1] Listar inventario          [5] Registrar venta
[2] Registrar nuevo producto   [6] Historial de ventas
[3] Buscar producto            [7] Reporte de inventario
[4] Actualizar producto        [8] Reporte de ventas
                                [9] Guardar cambios

[0] Salir del sistema

=====
Seleccione una opcion [0-9]: 5

=====
TECNOSTORE E.I.R.L.                      12/09/2026 00:00
REGISTRO DE VENTA
=====

ID      PRODUCTO                  CATEGORIA      PRECIO S/   STOCK   ESTADO
-----
P001    Audifonos Bluetooth ANC     Audio           159.90      14      OK
P002    Mouse Inalambrico 2.4G          Computo         49.90      22      OK
P003    Teclado Mecanico RGB            Computo        229.00       8      OK
P004    Cable HDMI 2.1 - 2m            Conectividad    35.50      40      OK
P005    Power Bank 20000 mAh            Energia        129.00       4      CRITICO
P006    Memoria USB 128 GB             Computo         69.90      17      OK
P007    Hub USB-C 7 en 1               Conectividad    119.00       6      OK
P008    Cargador GaN 65W               Energia        149.00      22      OK
P009    Parlante Bluetooth 20W          Audio           189.00       9      OK
P010    Soporte Laptop Aluminio         Accesorios      89.90      12      OK
P011    Webcam Full HD 1080p           Computo        189.50      15      OK
=====

Total de productos listados: 11

Nombre del cliente [ENTER = CLIENTE VARIOS]: Maria Quispe Ruiz

Agregue los productos de la venta (escriba F para finalizar).

ID del producto (F = finalizar): P099
[!] El producto 'P099' no existe en el catalogo.

ID del producto (F = finalizar): P002
Cantidad (stock disponible 22): seis
[!] Dato invalido: 'seis' no es un numero entero.
Cantidad (stock disponible 22): 6
[OK] Agregado: 6 x Mouse Inalambrico 2.4G = S/ 275.45 (dscto 8.0%)

ID del producto (F = finalizar): P004
Cantidad (stock disponible 40): 80
[!] Stock insuficiente: solo quedan 40 unidad(es)

ID del producto (F = finalizar): P004
Cantidad (stock disponible 40): 12
[OK] Agregado: 12 x Cable HDMI 2.1 - 2m = S/ 374.88 (dscto 12.0%)

ID del producto (F = finalizar): P011
Cantidad (stock disponible 15): 1
[OK] Agregado: 1 x Webcam Full HD 1080p = S/ 189.50 (dscto 0.0%)

ID del producto (F = finalizar): F
```

Figura 8. Opción 5: carga de los productos de la venta con sus validaciones.

La boleta B0001 consolida los tres ítems cargados. El sistema calculó el subtotal (S/ 839.83), el IGV del 18 % (S/ 151.17) y el total (S/ 991.00); a continuación agregó la venta al historial en modo a y regrabó el inventario en modo w.

```
C:\TecnoStore> python sistema_inventario.py

+-----+
|                     TECNOSTORE E.I.R.L.                     |
|                     BOLETA DE VENTA ELECTRONICA  B0001       |
+-----+
Fecha : 12/09/2026 00:00:06
Cliente: Maria Quispe Ruiz
+-----+
ID  DESCRIPCION      CANT  P.UNIT  DSCTO%  IMPORTE
+-----+
P002 Mouse Inalambrico 2.4G      6    49.90    8.0    275.45
P004 Cable HDMI 2.1 - 2m    12    35.50   12.0    374.88
P011 Webcam Full HD 1080p    1    189.50    0.0    189.50
+-----+
                                SUBTOTAL S/      839.83
                                IGV (18%) S/      151.17
                                TOTAL S/      991.00
+-----+

[i] Operacion de registro de venta finalizada.
[OK] Inventario grabado en 'datos/inventario.txt' (11 registro/s).
[i] Operacion de guardado finalizada.
[i] Transaccion finalizada.

Presione ENTER para volver al menu principal...
```

Figura 9. Opción 5: comprobante de venta emitido con subtotal, IGV y total.

El historial muestra cada boleta con su fecha, cliente, cantidad de ítems y total, junto con el acumulado de unidades vendidas y recaudación.

```
C:\TecnoStore> python sistema_inventario.py

=====
SISTEMA DE GESTION DE INVENTARIO Y VENTAS
TECNOSTORE E.I.R.L. - Venta de accesorios tecnologicos
=====
GESTION DE INVENTARIO      VENTAS Y REPORTES
-----
[1] Listar inventario      [5] Registrar venta
[2] Registrar nuevo producto [6] Historial de ventas
[3] Buscar producto        [7] Reporte de inventario
[4] Actualizar producto    [8] Reporte de ventas
[0] Salir del sistema      [9] Guardar cambios

=====
Seleccione una opcion [0-9]: 6

=====
TECNOSTORE E.I.R.L.                      12/09/2026 00:00
HISTORIAL DE VENTAS
=====
BOLETA  FECHA      CLIENTE      ITEMS  TOTAL S/
-----
B0001   12/09/2026 00:00:06  Maria Quispe Ruiz      3      991.00
-----
Comprobantes: 1  Unidades vendidas: 19  Recaudado: S/ 991.00

Presione ENTER para volver al menu principal...
```

Figura 10. Opción 6: historial de comprobantes emitidos.

El reporte entrega los indicadores de existencias, el resumen por categoría y la lista de productos que requieren reposición urgente, información que la gestión manual no permitía conocer.

```
C:\TecnoStore> python sistema_inventario.py

=====
SISTEMA DE GESTION DE INVENTARIO Y VENTAS
TECNOSTORE E.I.R.L. - Venta de accesorios tecnologicos
=====

GESTION DE INVENTARIO          VENTAS Y REPORTES
-----
[1] Listar inventario          [5] Registrar venta
[2] Registrar nuevo producto   [6] Historial de ventas
[3] Buscar producto            [7] Reporte de inventario
[4] Actualizar producto        [8] Reporte de ventas
[0] Salir del sistema          [9] Guardar cambios
=====

Seleccione una opcion [0-9]: 7

=====
TECNOSTORE E.I.R.L.                      12/09/2026 00:00
REPORTE DE INVENTARIO
=====

Productos registrados .....: 11
Unidades totales en stock ...: 150
Valorizado del inventario ...: S/ 16,992.10
Productos en stock critico ...: 1 (umbral 5 unidades)
Productos agotados .....: 0

RESUMEN POR CATEGORIA
-----
CATEGORIA      PRODUCTOS  UNIDADES  VALORIZADO S/
-----
Audio           2          23        3,939.60
Computo         4          55        6,471.70
Conectividad    2          34        1,708.00
Energia         2          26        3,794.00
Accesorios      1          12        1,078.80
-----

ALERTA: PRODUCTOS QUE REQUIEREN REPOSICION
-----
ID  PRODUCTO                CATEGORIA  PRECIO S/  STOCK  ESTADO
-----
P005  Power Bank 20000 mAh      Energia    129.00    4      CRITICO
-----

Total de productos listados: 1

Presione ENTER para volver al menu principal...
```

Figura 11. Opción 7: reporte de inventario con alerta de reposición.

Se presentan las ventas del día, el acumulado histórico, el ticket promedio y los productos más vendidos: el rendimiento diario real que el administrador necesitaba para tomar decisiones.

```

C:\TecnoStore> python sistema_inventario.py

=====
SISTEMA DE GESTION DE INVENTARIO Y VENTAS
TECNOSTORE E.I.R.L. - Venta de accesorios tecnologicos
=====
GESTION DE INVENTARIO          VENTAS Y REPORTES
-----
[1] Listar inventario          [5] Registrar venta
[2] Registrar nuevo producto   [6] Historial de ventas
[3] Buscar producto            [7] Reporte de inventario
[4] Actualizar producto        [8] Reporte de ventas
[0] Salir del sistema          [9] Guardar cambios
=====
Seleccione una opcion [0-9]: 8

=====
TECNOSTORE E.I.R.L.                  12/09/2026 00:00
REPORTE DE VENTAS
=====
VENTAS DEL DIA 12/09/2026
-----
Comprobantes emitidos .....: 1
Unidades vendidas .....: 19
Venta neta (sin IGV) .....: S/ 839.83
IGV del dia .....: S/ 151.17
Total recaudado .....: S/ 991.00
Ticket promedio .....: S/ 991.00

ACUMULADO HISTORICO
-----
Comprobantes .....: 1
Total recaudado .....: S/ 991.00
Ticket promedio .....: S/ 991.00

PRODUCTOS MAS VENDIDOS
-----
ID      PRODUCTO                      UNIDADES
-----
P004    Cable HDMI 2.1 - 2m             12
P002    Mouse Inalambrico 2.4G             6
P011    Webcam Full HD 1080p                1
=====

Presione ENTER para volver al menu principal...

```

Figura 12. Opción 8: reporte de ventas del día y ranking de productos.

La opción 9 invoca `guardar_inventario()` en modo `w` y confirma que la información quedará disponible en la próxima ejecución.

```

C:\TecnoStore> python sistema_inventario.py

=====
SISTEMA DE GESTION DE INVENTARIO Y VENTAS
TECNOSTORE E.I.R.L. - Venta de accesorios tecnologicos
=====
GESTION DE INVENTARIO          VENTAS Y REPORTES
-----
[1] Listar inventario          [5] Registrar venta
[2] Registrar nuevo producto   [6] Historial de ventas
[3] Buscar producto            [7] Reporte de inventario
[4] Actualizar producto        [8] Reporte de ventas
[0] Salir del sistema          [9] Guardar cambios
=====
Seleccione una opcion [0-9]: 9

=====
TECNOSTORE E.I.R.L.                  12/09/2026 00:00
GUARDADO DE INFORMACION
=====
[OK] Inventario grabado en 'datos/inventario.txt' (11 registro/s).
[i] Operación de guardado finalizada.

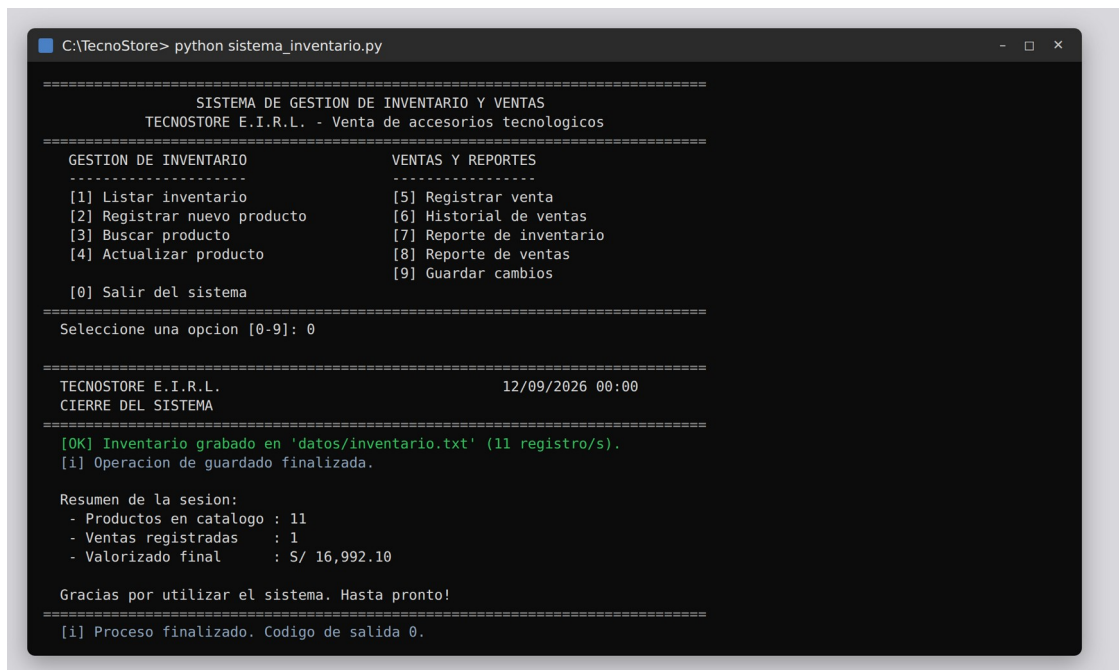
[OK] La informacion quedara disponible en la proxima ejecucion.

Presione ENTER para volver al menu principal...

```

Figura 13. Opción 9: grabado manual del inventario en el archivo de texto.

La salida del bucle `while` mediante `break` ejecuta un guardado de seguridad, muestra el resumen de la sesión y finaliza el proceso de forma ordenada.



```
C:\TecnoStore> python sistema_inventario.py

=====
SISTEMA DE GESTION DE INVENTARIO Y VENTAS
TECNOSTORE E.I.R.L. - Venta de accesorios tecnologicos
=====

GESTION DE INVENTARIO          VENTAS Y REPORTES
-----
[1] Listar inventario           [5] Registrar venta
[2] Registrar nuevo producto    [6] Historial de ventas
[3] Buscar producto             [7] Reporte de inventario
[4] Actualizar producto         [8] Reporte de ventas
                                [9] Guardar cambios

[0] Salir del sistema

=====
Seleccione una opcion [0-9]: 0

=====
TECNOSTORE E.I.R.L.              12/09/2026 00:00
CIERRE DEL SISTEMA
=====

[OK] Inventario grabado en 'datos/inventario.txt' (11 registro/s).
[i] Operacion de guardado finalizada.

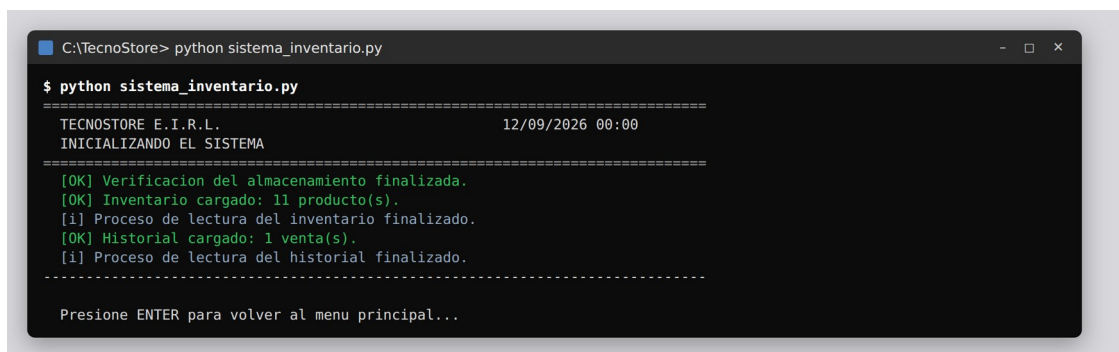
Resumen de la sesion:
- Productos en catalogo : 11
- Ventas registradas    : 1
- Valorizado final      : S/ 16,992.10

Gracias por utilizar el sistema. Hasta pronto!

=====
[i] Proceso finalizado. Codigo de salida 0.
```

Figura 14. Opción 0: cierre controlado del sistema con resumen de la sesión.

Al volver a ejecutar el programa, el sistema recupera los 11 productos y la venta registrada en la sesión anterior. Esto demuestra que los datos se conservan entre ejecuciones, tal como exige el caso práctico.



```
C:\TecnoStore> python sistema_inventario.py

$ python sistema_inventario.py

=====
TECNOSTORE E.I.R.L.              12/09/2026 00:00
INICIALIZANDO EL SISTEMA
=====

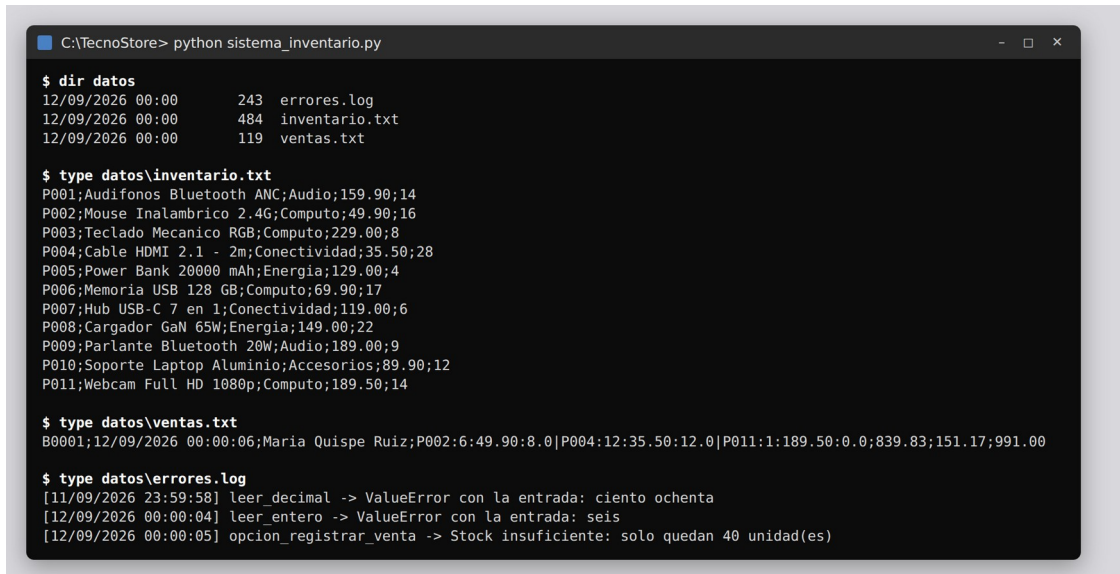
[OK] Verificacion del almacenamiento finalizada.
[OK] Inventario cargado: 11 producto(s).
[i] Proceso de lectura del inventario finalizado.
[OK] Historial cargado: 1 venta(s).
[i] Proceso de lectura del historial finalizado.

=====

Presione ENTER para volver al menu principal...
```

Figura 15. Verificación de la persistencia: segunda ejecución del sistema.

Se aprecian los tres archivos de la carpeta `datos/`: el catálogo con el stock ya descontado por la venta (P002 pasó de 22 a 16 y P004 de 40 a 28), el historial con el comprobante B0001 y la bitácora con las tres incidencias capturadas durante la sesión.



```
C:\TecnoStore> python sistema_inventario.py

$ dir datos
12/09/2026 00:00      243 errores.log
12/09/2026 00:00      484 inventario.txt
12/09/2026 00:00      119 ventas.txt

$ type datos\inventario.txt
P001;Audifonos Bluetooth ANC;Audio;159.90;14
P002;Mouse Inalambrico 2.4G;Computo;49.90;16
P003;Teclado Mecanico RGB;Computo;229.00;8
P004;Cable HDMI 2.1 - 2m;Conectividad;35.50;28
P005;Power Bank 20000 mAh;Energia;129.00;4
P006;Memoria USB 128 GB;Computo;69.90;17
P007;Hub USB-C 7 en 1;Conectividad;119.00;6
P008;Cargador GaN 65W;Energia;149.00;22
P009;Parlante Bluetooth 20W;Audio;189.00;9
P010;Soporte Laptop Aluminio;Accesorios;89.90;12
P011;Webcam Full HD 1080p;Computo;189.50;14

$ type datos\ventas.txt
B0001;12/09/2026 00:00:06;Maria Quispe Ruiz;P002:6:49.90:8.0|P004:12:35.50:12.0|P011:1:189.50:0.0;839.83;151.17;991.00

$ type datos\errores.log
[11/09/2026 23:59:58] leer_decimal -> ValueError con la entrada: ciento ochenta
[12/09/2026 00:00:04] leer_entero -> ValueError con la entrada: seis
[12/09/2026 00:00:05] opcion_registrar_venta -> Stock insuficiente: solo quedan 40 unidad(es)
```

Figura 16. Contenido de los archivos de texto generados por el sistema.

6.6. Pruebas realizadas

Se ejecutaron dieciséis casos de prueba orientados a verificar el cumplimiento de las condiciones del caso práctico, con especial énfasis en las entradas incorrectas, que son la causa más frecuente de cierre inesperado de un programa de consola.

N.º	CASO DE PRUEBA	DATO INGRESADO	RESULTADO OBTENIDO	EVIDENCIA
1	Carga inicial sin archivos de datos	–	Informa la ausencia, crea los archivos y arranca con inventario vacío.	Figura 1
2	Carga inicial con archivos existentes	–	Recupera 11 productos y 1 venta desde el disco.	Figura 15
3	Opción de menú inválida	8x · 99 · vacío	Rechaza cada opción y vuelve a solicitarla sin salir del menú.	Figura 17
4	Precio no numérico	ciento ochenta	Captura <code>ValueError</code> , avisa y vuelve a pedir el precio.	Figura 4
5	Stock negativo	-8	Rechaza el valor por rango e informa el mínimo permitido.	Figura 4
6	Identificador duplicado	P003	Bloquea el alta y muestra "El ID ya existe".	Figura 5
7	Código de producto inexistente en venta	P099	Avisa y permite continuar la venta con continúe.	Figura 8
8	Cantidad no numérica en venta	seis	Captura <code>ValueError</code> y vuelve a solicitar la cantidad.	Figura 8
9	Cantidad mayor al stock disponible	80 (stock 40)	Rechaza el ítem con "Stock insuficiente" y mantiene la venta activa.	Figura 8
10	Descuento por volumen	6 y 12 unidades	Aplica 8 % y 12 % respectivamente, según la escala definida.	Figura 9
11	Cálculo de IGV y total	Subtotal S/ 839.83	IGV S/ 151.17 y total S/ 991.00 (18 % exacto).	Figura 9
12	Actualización de stock por ingreso	+20 unidades	Stock de 2 a 22 y grabado inmediato del archivo.	Figura 7
13	Descuento de stock por venta	Venta de 6 y 12 unidades	P002 de 22 a 16 y P004 de 40 a 28 en <code>inventario.txt</code> .	Figura 16
14	Persistencia del historial de ventas	Boleta B0001	El comprobante se agregó al final de <code>ventas.txt</code> (modo a).	Figura 16
15	Registro de incidencias	3 errores provocados	Las tres excepciones quedaron registradas en <code>errores.log</code> con fecha y origen.	Figura 16
16	Cierre controlado del sistema	Opción 0	Guarda el inventario, muestra el resumen y finaliza con código de salida 0.	Figura 14

La Figura 17 evidencia el caso 3: el menú rechaza una opción con letras, una fuera de rango y una entrada vacía, y permanece activo hasta que el usuario digita una opción válida.


```
C:\TecnoStore> python sistema_inventario.py

=====
SISTEMA DE GESTION DE INVENTARIO Y VENTAS
TECNOSTORE E.I.R.L. - Venta de accesorios tecnologicos
=====

GESTION DE INVENTARIO          VENTAS Y REPORTES
-----
[1] Listar inventario          [5] Registrar venta
[2] Registrar nuevo producto  [6] Historial de ventas
[3] Buscar producto           [7] Reporte de inventario
[4] Actualizar producto        [8] Reporte de ventas
                                [9] Guardar cambios
[0] Salir del sistema

=====
Seleccione una opcion [0-9]: 8x
[!] Opcion '8x' no valida. Elija una opcion del menu.
Seleccione una opcion [0-9]: 99
[!] Opcion '99' no valida. Elija una opcion del menu.
Seleccione una opcion [0-9]:
[!] Opcion '' no valida. Elija una opcion del menu.
Seleccione una opcion [0-9]: 0

=====
TECNOSTORE E.I.R.L.                  12/09/2026 00:18
CIERRE DEL SISTEMA

=====
[OK] Inventario grabado en 'datos/inventario.txt' (11 registro/s).
[i] Operacion de guardado finalizada.

Resumen de la sesion:
- Productos en catalogo : 11
- Ventas registradas    : 1
- Valorizado final      : S/ 16,992.10

Gracias por utilizar el sistema. Hasta pronto!

=====
[i] Proceso finalizado.Codigo de salida 0.
```

Figura 17. Validación de la opción del menú principal (caso de prueba 3).

Resultado global: 16 casos ejecutados, 16 conformes. En ninguna de las pruebas la aplicación se cerró de manera inesperada ni mostró un mensaje de error propio del intérprete de Python, que era precisamente el riesgo que el caso práctico exigía eliminar.

7. NORMAS TÉCNICAS, SEGURIDAD Y SALUD EN EL TRABAJO Y MEDIO AMBIENTE

El objetivo del curso exige desarrollar la competencia **cumpliendo las normas técnicas, las normas de seguridad y salud en el trabajo y actuando de manera responsable con el medio ambiente**. A continuación se detalla cómo se atendió cada aspecto durante el desarrollo del trabajo.

7.1. Normas técnicas aplicadas

NORMA O ESTÁNDAR	APLICACIÓN EN EL TRABAJO
PEP 8 — Guía de estilo del código Python	Nombres de funciones y variables en minúsculas con guion bajo, constantes en mayúsculas, líneas de hasta 79 caracteres, cuatro espacios de indentación y dos líneas en blanco entre funciones.
PEP 257 — Convención de docstrings	Las 51 funciones documentan su propósito, sus parámetros, su valor de retorno y las excepciones que lanzan.
Codificación UTF-8	Todos los archivos de datos se abren indicando <code>encoding="utf-8"</code> , evitando la pérdida de caracteres del idioma español.
Principio de responsabilidad única	Cada función resuelve una sola tarea; la lógica de negocio se mantiene separada de la interfaz de consola.
Manejo estructurado de excepciones	Captura de excepciones específicas, registro en bitácora y continuidad de la ejecución.

7.2. Seguridad y salud en el trabajo

RIESGO DEL PUESTO DE TRABAJO	MEDIDA ADOPTADA
Fatiga visual por exposición prolongada a la pantalla	Aplicación de la regla 20-20-20 durante la programación (cada 20 minutos, mirar a 6 metros durante 20 segundos) y uso de un tamaño de fuente legible en la consola.
Trastornos musculoesqueléticos por postura inadecuada	Puesto de trabajo con la pantalla a la altura de los ojos, espalda apoyada, antebrazos horizontales y pausas activas cada hora.
Estrés y sobrecarga mental	Planificación del trabajo en cinco etapas (análisis, diseño, codificación, pruebas y documentación), evitando la acumulación de tareas al final del plazo.
Pérdida de información del usuario final	El sistema graba los datos tras cada operación y registra las incidencias en una bitácora, lo que reduce el estrés del operador ante un corte imprevisto.
Riesgo eléctrico en el puesto	Uso de tomacorrientes con conexión a tierra y supresor de picos; no se ingieren alimentos ni bebidas junto al equipo.

7.3. Responsabilidad ambiental

- **Reducción del consumo de papel:** la solución sustituye los cuadernos de registro y la impresión de listados por archivos digitales, y el comprobante se visualiza en pantalla antes de decidir si se imprime.

- **Eficiencia energética del software:** el programa no ejecuta procesos en segundo plano ni consultas innecesarias; solo trabaja cuando el operador lo solicita, lo que reduce el consumo del equipo.
- **Bajo requerimiento de hardware:** al no necesitar librerías externas ni un servidor de base de datos, la aplicación funciona en equipos antiguos, prolongando su vida útil y retrasando la generación de residuos electrónicos (RAEE).
- **Archivos livianos:** el uso de texto plano mantiene el almacenamiento por debajo de 1 MB, frente a los archivos de hojas de cálculo que la empresa utilizaba.
- **Gestión responsable de residuos:** durante el desarrollo se evitó la impresión de borradores y los equipos en desuso se derivan a los puntos de acopio autorizados de RAEE.

8. CONCLUSIONES

Primera. Se desarrolló una aplicación de consola en Python que automatiza el registro, la consulta y la actualización del inventario, así como el procesamiento de las transacciones de venta de la microempresa, resolviendo los cinco problemas identificados en el diagnóstico: la desactualización del stock, los errores humanos de cálculo, la lentitud en la búsqueda de productos, el desconocimiento del rendimiento diario y el riesgo de pérdida de información.

Segunda. La programación modular demostró ser determinante para la calidad del resultado. Distribuir 51 funciones en cinco capas con responsabilidad única evitó la programación lineal o monolítica y produjo un código en el que la lógica de negocio no depende de la consola, por lo que puede reutilizarse en una interfaz gráfica o web sin modificación alguna.

Tercera. El uso de estructuras de datos nativas —lista de diccionarios para el inventario y diccionario anidado para las ventas— permitió modelar la información con nombres de campo significativos, realizar búsquedas y actualizaciones directas sobre la referencia obtenida y generar reportes mediante comprensiones de listas, sin recurrir a estructuras externas.

Cuarta. El manejo de excepciones con bloques `try-except-else-finally` cumplió su propósito: en los dieciséis casos de prueba ejecutados, incluidas entradas no numéricas, cantidades negativas, identificadores duplicados y cantidades superiores al stock, la aplicación nunca se cerró de forma inesperada ni mostró un mensaje de error del intérprete al usuario final.

Quinta. La persistencia mediante context managers (`with`) y los modos `r`, `w` y `a` garantizó que la información se conserve entre ejecuciones. La verificación documentada en las Figuras 15 y 16 confirma que el sistema recupera el catálogo con el stock ya descontado y el historial de ventas completo al volver a iniciarse.

Sexta. El menú interactivo controlado por `while` e `if-elif-else`, junto con la validación permanente de las entradas y los mensajes en lenguaje sencillo, hace que el sistema pueda ser operado por personal sin conocimientos técnicos avanzados, tal como exigía el caso práctico.

Séptima. El trabajo permitió integrar y evidenciar los contenidos curriculares del curso —tipos de datos, estructuras condicionales y repetitivas, estructuras de datos, funciones, excepciones y archivos— en una solución única y funcional, aplicando además las normas técnicas de codificación, las normas de seguridad y salud en el trabajo y criterios de responsabilidad ambiental.

9. RECOMENDACIONES

9.1. Para la implementación en la microempresa

- **Capacitar al personal** en el uso del sistema durante una sesión breve, apoyándose en el manual de la sección 6.3 y en las capturas de pantalla de la sección 6.5.
- **Realizar un inventario físico inicial** antes de cargar el catálogo, para que el stock registrado coincida exactamente con el stock real desde el primer día.
- **Establecer una copia de respaldo diaria** de la carpeta `datos/` en una unidad externa o en un servicio en la nube, ya que los archivos de texto son el único repositorio de la información.
- **Revisar semanalmente** el reporte de stock crítico (opción 7) para programar las reposiciones antes de que un producto se agote.
- **Consultar periódicamente** el archivo `datos/errores.log`, que permite detectar dificultades recurrentes del operador y reforzar su capacitación.

9.2. Para futuras versiones del sistema

MEJORA PROPUESTA	BENEFICIO ESPERADO	COMPLEJIDAD
Indexar el inventario en un diccionario <code>{id: producto}</code>	Búsqueda directa $O(1)$, útil si el catálogo supera varios miles de productos.	Baja
Migrar el formato a CSV con el módulo <code>csv</code>	Compatibilidad directa con hojas de cálculo y manejo automático de delimitadores.	Baja
Incorporar control de usuarios y contraseñas	Trazabilidad de quién registró cada venta y restricción de la opción de baja de productos.	Media
Migrar la persistencia a SQLite	Consultas más potentes, integridad referencial y acceso simultáneo de varios equipos.	Media
Añadir gestión de compras y proveedores	Cierre del ciclo completo de abastecimiento y cálculo del margen real por producto.	Media
Desarrollar una interfaz gráfica con <code>tkinter</code>	Operación más intuitiva; la capa de lógica de negocio se reutiliza sin cambios.	Alta
Exportar reportes a PDF o Excel	Entrega de información al contador y a la gerencia en formatos de uso común.	Alta

10. REFERENCIAS BIBLIOGRÁFICAS

- Python Software Foundation. (2024). *The Python Tutorial — Python 3 documentation*. <https://docs.python.org/es/3/tutorial/>
- Python Software Foundation. (2024). *Errors and Exceptions — Python 3 documentation*. <https://docs.python.org/es/3/tutorial/errors.html>
- Python Software Foundation. (2024). *Reading and Writing Files — Python 3 documentation*. <https://docs.python.org/es/3/tutorial/inputoutput.html>
- Van Rossum, G., Warsaw, B. y Coghlan, N. (2013). *PEP 8 — Style Guide for Python Code*. Python Software Foundation. <https://peps.python.org/pep-0008/>
- Goodger, D. y Van Rossum, G. (2001). *PEP 257 — Docstring Conventions*. Python Software Foundation. <https://peps.python.org/pep-0257/>
- Matthes, E. (2023). *Python Crash Course: A Hands-On, Project-Based Introduction to Programming* (3.ª ed.). No Starch Press.
- Joyanes Aguilar, L. (2013). *Fundamentos generales de programación*. McGraw-Hill Interamericana.
- Cairó Battistutti, O. (2005). *Metodología de la programación: algoritmos, diagramas de flujo y programas* (3.ª ed.). Alfaomega.
- Superintendencia Nacional de Aduanas y de Administración Tributaria. (2024). *Impuesto General a las Ventas (IGV)*. <https://www.sunat.gob.pe/>
- Ministerio de Trabajo y Promoción del Empleo. (2011). *Ley N.º 29783, Ley de Seguridad y Salud en el Trabajo*. Diario Oficial El Peruano.

ANEXO A. CÓDIGO FUENTE COMPLETO

Se transcribe íntegramente el archivo ``sistema_inventario.py``, entregable principal del trabajo (1 538 líneas, 51 funciones documentadas). El archivo se adjunta además en formato digital, listo para ejecutarse.

```
# -*- coding: utf-8 -*-
"""
=====
SISTEMA DE GESTION DE INVENTARIO Y REGISTRO DE VENTAS PARA UNA MICROEMPRESA
Aplicacion de consola desarrollada en Python 3
=====
Curso      : PIAD-221 - Algoritmos y Programacion para Desarrollo de Software
Carrera    : Tecnologias de la Informacion
Trabajo    : Trabajo Final de Curso
Caso       : Tienda de accesorios tecnologicos "TecnoStore"
Estudiante : [COLOCAR NOMBRES Y APELLIDOS]
ID          : [COLOCAR ID / CODIGO DE MATRICULA]
Instructor : [COLOCAR NOMBRE DEL INSTRUCTOR]
Fecha      : [COLOCAR FECHA DE ENTREGA]
=====
DESCRIPCION GENERAL
=====
Aplicacion de consola que automatiza el registro, consulta y actualizacion de
productos del inventario, asi como el procesamiento de transacciones de venta
de una microempresa dedicada a la comercializacion de accesorios tecnologicos.

La solucion sustituye el registro manual en cuadernos y hojas de calculo
desconectadas, eliminando la desactualizacion del stock, los errores humanos
de calculo y la perdida de tiempo en la busqueda de productos.
=====
ARQUITECTURA MODULAR (5 capas, sin programacion lineal ni monolitica)
=====
CAPA 1 - CONFIGURACION .... Constantes globales del sistema.
CAPA 2 - PERSISTENCIA ..... Lectura/escritura en archivos de texto plano
                             con la sentencia "with" y los modos r, w, a.
CAPA 3 - VALIDACION ..... Entrada segura de datos con try-except-else-finally.
CAPA 4 - LOGICA DE NEGOCIO Reglas del negocio; NO usa print() ni input().
CAPA 5 - INTERFAZ / CONTROL Presentacion en consola y menu principal.

Regla de separacion de responsabilidades: la CAPA 4 (logica de negocio) es
completamente independiente de la consola; recibe parametros y devuelve
valores de retorno, por lo que puede reutilizarse en una futura interfaz
grafica o web sin modificar una sola linea.
=====
ARCHIVOS DE DATOS GENERADOS (persistencia entre ejecuciones)
=====
datos/inventario.txt ... Catalogo de productos (se reescribe con modo "w").
datos/ventas.txt ..... Historial de ventas (se agrega con modo "a").
datos/errores.log ..... Bitacora de excepciones (se agrega con modo "a").
=====
EJECUCION
=====
python sistema_inventario.py
=====
"""

import os
import sys
from datetime import datetime

# =====
# CAPA 1: CONFIGURACION - CONSTANTES GLOBALES
# =====
# Las constantes se declaran en el ambito GLOBAL (accesibles desde cualquier
# funcion en modo lectura). Las variables de trabajo, en cambio, son LOCALES y
# viajan entre funciones como parametros y valores de retorno.
# =====

NOMBRE_EMPRESA = "TECNOSTORE E.I.R.L."
RUBRO_EMPRESA = "Venta de accesorios tecnologicos"

CARPETA_DATOS = "datos" # Carpeta de persistencia
ARCHIVO_INVENTARIO = os.path.join(CARPETA_DATOS, "inventario.txt")
ARCHIVO_VENTAS = os.path.join(CARPETA_DATOS, "ventas.txt")
ARCHIVO_LOG = os.path.join(CARPETA_DATOS, "errores.log")

SEPARADOR_CAMPO = ";" # Separa los campos de un registro
SEPARADOR_ITEM = "|" # Separa los items dentro de una venta
SEPARADOR_SUBCAMPO = ":" # Separa los datos dentro de un item

IGV = 0.18 # Impuesto General a las Ventas (18%)
STOCK_MINIMO = 5 # Umbral de alerta de stock critico
ANCHO = 78 # Ancho estandar de la consola

# Reglas de descuento por volumen: (cantidad minima, porcentaje de descuento)
ESCALA_DESCUENTOS = [(12, 12.0), (6, 8.0), (3, 5.0)]
```



```

CATEGORIAS = ["Audio", "Computo", "Conectividad", "Energia", "Accesorios"]

# Opciones validas del menu principal (se usan para validar la navegacion)
OPCIONES_MENU = ["1", "2", "3", "4", "5", "6", "7", "8", "9", "0"]

# =====
# CAPA 2: PERSISTENCIA DE DATOS
# =====
# Toda la entrada/salida a disco se concentra aqui. Se utiliza EXCLUSIVAMENTE
# archivos de texto plano manejados con la sentencia "with" (context manager),
# que garantiza el cierre automatico del archivo incluso si ocurre un error.
#
# Modo "r" -> leer el catalogo y el historial al iniciar el programa.
# Modo "w" -> reescribir por completo el inventario actualizado.
# Modo "a" -> agregar (append) una venta o un error sin borrar lo anterior.
# =====

def registrar_error(origen, detalle):
    """Escribe una excepcion en la bitacora datos/errores.log (modo "a").

    Parametros:
        origen (str): nombre de la funcion donde ocurrio el error.
        detalle (str): descripcion tecnica de la excepcion capturada.
    Retorna:
        bool: True si el error pudo registrarse en la bitacora.
    """
    try:
        os.makedirs(CARPETA_DATOS, exist_ok=True)
        marca = datetime.now().strftime("%d/%m/%Y %H:%M:%S")
        with open(ARCHIVO_LOG, "a", encoding="utf-8") as bitacora:
            bitacora.write("[{0}] {1} -> {2}\n".format(marca, origen, detalle))
    except OSError:
        # Si ni siquiera se puede escribir la bitacora, el programa NO se detiene.
        return False
    else:
        return True

def inicializar_almacenamiento():
    """Crea la carpeta y los archivos de datos si aun no existen.

    Se ejecuta una sola vez al arrancar el sistema. Utiliza el modo "a", que
    crea el archivo cuando no existe y no destruye la informacion si ya existe.

    Retorna:
        bool: True si el almacenamiento quedo listo para usarse.
    """
    listo = False
    try:
        os.makedirs(CARPETA_DATOS, exist_ok=True)
        for ruta in (ARCHIVO_INVENTARIO, ARCHIVO_VENTAS, ARCHIVO_LOG):
            with open(ruta, "a", encoding="utf-8"):
                pass # Solo se asegura la existencia fisica del archivo.
    except PermissionError as error:
        print(" [!] Sin permisos de escritura en la carpeta de datos:", error)
        registrar_error("inicializar_almacenamiento", str(error))
    except OSError as error:
        print(" [!] No fue posible preparar el almacenamiento:", error)
        registrar_error("inicializar_almacenamiento", str(error))
    else:
        listo = True
    finally:
        # El bloque finally SIEMPRE se ejecuta: confirme o no la operacion.
        print(" [OK] Verificacion del almacenamiento finalizada.")
    return listo

def linea_a_producto(linea):
    """Convierte una linea del archivo de texto en un diccionario producto.

    Formato del registro: id;nombre;categoria;precio;stock

    Parametros:
        linea (str): linea leida del archivo inventario.txt.
    Retorna:
        dict: producto estructurado.
    Lanza:
        ValueError: si el registro esta incompleto o mal formado.
    """
    campos = linea.strip().split(SEPARADOR_CAMPO)
    if len(campos) != 5:
        raise ValueError("El registro no tiene los 5 campos requeridos")

    producto = {
        "id": campos[0].strip().upper(),
        "nombre": campos[1].strip(),
        "categoria": campos[2].strip(),
        "precio": float(campos[3]), # Puede lanzar ValueError
        "stock": int(campos[4]), # Puede lanzar ValueError
    }
    if producto["precio"] < 0 or producto["stock"] < 0:
        raise ValueError("El precio y el stock no pueden ser negativos")
    return producto

```

```

def producto_a_linea(producto):
    """Convierte un diccionario producto en una linea de texto grabable.

    Parametros:
        producto (dict): producto del inventario.
    Retorna:
        str: linea con el formato id;nombre;categoria;precio;stock
    """
    return "{0};{1};{2};{3:.2f};{4}\n".format(
        producto["id"], producto["nombre"], producto["categoria"],
        producto["precio"], producto["stock"])

def cargar_inventario(ruta=ARCHIVO_INVENTARIO):
    """Carga el catalogo de productos desde el archivo de texto (modo "r").

    Los registros corruptos NO detienen la carga: se informan, se registran en
    la bitacora y el proceso continua con el resto de lineas (continue).

    Parametros:
        ruta (str): ubicacion del archivo de inventario.
    Retorna:
        list: lista de diccionarios con los productos cargados.
    """
    inventario = []
    try:
        with open(ruta, "r", encoding="utf-8") as archivo:
            contenido = archivo.read() # Metodo read()
    except FileNotFoundError:
        print(" [!] No se encontro '{0}'. Se iniciara un inventario vacio.".format(ruta))
        registrar_error("cargar_inventario", "FileNotFoundError: " + ruta)
    except PermissionError as error:
        print(" [!] Archivo de inventario bloqueado por el sistema:", error)
        registrar_error("cargar_inventario", str(error))
    except UnicodeDecodeError as error:
        print(" [!] El archivo de inventario tiene una codificacion invalida.")
        registrar_error("cargar_inventario", str(error))
    else:
        numero = 0
        for linea in contenido.splitlines():
            numero += 1
            if not linea.strip():
                continue # Ignora lineas en blanco
            try:
                producto = linea_a_producto(linea)
            except ValueError as error:
                print(" [!] Linea {0} descartada del inventario ({1}).".format(numero, error))
                registrar_error("cargar_inventario", "Linea {0}: {1}".format(numero, error))
                continue # El programa NO se interrumpe
            inventario.append(producto)
        print(" [OK] Inventario cargado: {0} producto(s).".format(len(inventario)))
    finally:
        print(" [i] Proceso de lectura del inventario finalizado.")
    return inventario

def guardar_inventario(inventario, ruta=ARCHIVO_INVENTARIO):
    """Graba TODO el inventario en el archivo de texto (modo "w").

    El modo "w" reescribe el archivo completo, de modo que las altas, bajas y
    modificaciones quedan reflejadas exactamente como estan en memoria.

    Parametros:
        inventario (list): lista de diccionarios con los productos.
        ruta (str) : archivo destino.
    Retorna:
        bool: True si la informacion se grabo correctamente.
    """
    grabado = False
    try:
        os.makedirs(CARPETA_DATOS, exist_ok=True)
        with open(ruta, "w", encoding="utf-8") as archivo:
            for producto in inventario:
                archivo.write(producto_a_linea(producto)) # Metodo write()
    except PermissionError as error:
        print(" [!] No se pudo grabar el inventario (permisos):", error)
        registrar_error("guardar_inventario", str(error))
    except OSError as error:
        print(" [!] Error de escritura en disco:", error)
        registrar_error("guardar_inventario", str(error))
    else:
        grabado = True
        print(" [OK] Inventario grabado en '{0}' ({1} registro/s).".format(ruta, len(inventario)))
    finally:
        print(" [i] Operacion de guardado finalizada.")
    return grabado

def venta_a_linea(venta):
    """Convierte una venta (diccionario anidado) en una linea de texto.

    Formato: boleta;fecha;cliente;items;subtotal;igv;total
    donde items = id:cantidad:precio:descuento|id:cantidad:precio:descuento

    Parametros:
        venta (dict): venta construida por construir_venta().
    Retorna:

```

```

    str: linea lista para grabarse en ventas.txt
    """
    detalle = []
    for item in venta["items"]:
        campos_item = [
            item["id"],
            str(item["cantidad"]),
            "{0:.2f}".format(item["precio_unitario"]),
            "{0:.1f}".format(item["descuento"]),
        ]
        detalle.append(SEPARADOR_SUBCAMPO.join(campos_item))
    return "{0};{1};{2};{3};{4:.2f};{5:.2f};{6:.2f}\n".format(
        venta["boleta"], venta["fecha"], venta["cliente"],
        SEPARADOR_ITEM.join(detalle),
        venta["subtotal"], venta["igv"], venta["total"])

def linea_a_venta(linea):
    """Convierte una linea de ventas.txt en un diccionario anidado de venta.

    Parametros:
        linea (str): registro leido del historial.
    Retorna:
        dict: venta con su lista interna de items.
    Lanza:
        ValueError: si el registro esta incompleto o mal formado.
    """
    campos = linea.strip().split(SEPARADOR_CAMPO)
    if len(campos) != 7:
        raise ValueError("El registro de venta no tiene los 7 campos requeridos")

    items = []
    for bloque in campos[3].split(SEPARADOR_ITEM):
        datos = bloque.split(SEPARADOR_SUBCAMPO)
        if len(datos) != 4:
            raise ValueError("Item de venta mal formado")
        items.append({
            "id": datos[0],
            "cantidad": int(datos[1]),
            "precio_unitario": float(datos[2]),
            "descuento": float(datos[3]),
        })

    return {
        "boleta": campos[0],
        "fecha": campos[1],
        "cliente": campos[2],
        "items": items,
        "subtotal": float(campos[4]),
        "igv": float(campos[5]),
        "total": float(campos[6]),
    }

def cargar_ventas(ruta=ARCHIVO_VENTAS):
    """Carga el historial de ventas desde el archivo de texto (modo "r").

    Parametros:
        ruta (str): ubicacion del archivo de ventas.
    Retorna:
        list: lista de diccionarios con las ventas registradas.
    """
    ventas = []
    try:
        with open(ruta, "r", encoding="utf-8") as archivo:
            lineas = archivo.readlines() # Metodo readlines()
    except FileNotFoundError:
        print(" [!] Aun no existe historial de ventas. Se creara al vender.")
        registrar_error("cargar_ventas", "FileNotFoundError: " + ruta)
    except OSError as error:
        print(" [!] No se pudo leer el historial de ventas:", error)
        registrar_error("cargar_ventas", str(error))
    else:
        for numero, linea in enumerate(lineas, start=1):
            if not linea.strip():
                continue
            try:
                ventas.append(linea_a_venta(linea))
            except ValueError as error:
                registrar_error("cargar_ventas", "Linea {0}: {1}".format(numero, error))
                continue
        print(" [OK] Historial cargado: {0} venta(s)".format(len(ventas)))
    finally:
        print(" [i] Proceso de lectura del historial finalizado.")
    return ventas

def registrar_venta_en_archivo(venta, ruta=ARCHIVO_VENTAS):
    """Agrega UNA venta al final del historial (modo "a" - append).

    A diferencia del inventario, el historial de ventas nunca se reescribe:
    cada transaccion se agrega al final para conservar la trazabilidad.

    Parametros:
        venta (dict): venta a registrar.
        ruta (str) : archivo destino.
    Retorna:

```

```

        bool: True si la venta quedo grabada en disco.
    """
    grabada = False
    try:
        os.makedirs(CARPETA_DATOS, exist_ok=True)
        with open(ruta, "a", encoding="utf-8") as archivo:
            archivo.write(venta_a_linea(venta)) # write() en modo append
    except PermissionError as error:
        print(" [!] No se pudo registrar la venta (permisos):", error)
        registrar_error("registrar_venta_en_archivo", str(error))
    except OSError as error:
        print(" [!] Error al grabar la venta:", error)
        registrar_error("registrar_venta_en_archivo", str(error))
    else:
        grabada = True
    finally:
        print(" [i] Operacion de registro de venta finalizada.")
    return grabada

# =====
# CAPA 3: VALIDACION Y ENTRADA SEGURA DE DATOS
# -----
# Ninguna funcion de este bloque permite que una entrada incorrecta del usuario
# cierre inesperadamente el programa. Cada lectura se protege con la estructura
# completa try-except-else-finally y con un bucle while que insiste hasta
# obtener un dato valido (o hasta que el usuario cancele con la palabra "X").
# =====

def leer_cadena(mensaje, obligatorio=True, maximo=40, cancelable=True):
    """Lee texto desde el teclado validando longitud y obligatoriedad.

    Parametros:
        mensaje (str) : indicacion mostrada al usuario.
        obligatorio(bool): si True, no se acepta una cadena vacia.
        maximo (int) : cantidad maxima de caracteres permitidos.
        cancelable (bool): si True, la letra X cancela la operacion.
    Retorna:
        str : texto validado, o None si el usuario cancela la operacion.
    """
    while True:
        try:
            dato = input(mensaje).strip()
        except (EOFError, KeyboardInterrupt):
            # Ctrl+C / Ctrl+Z no deben provocar un cierre abrupto con traceback.
            print("\n [i] Entrada interrumpida por el usuario.")
            return None
        except ValueError as error:
            registrar_error("leer_cadena", str(error))
            continue

        if cancelable and dato.upper() == "X":
            return None
        if not dato and obligatorio:
            print(" [!] El dato es obligatorio. Intente nuevamente.")
            continue
        if len(dato) > maximo:
            print(" [!] Maximo {0} caracteres. Intente nuevamente.".format(maximo))
            continue
        return dato

def leer_entero(mensaje, minimo=None, maximo=None, cancelable=True):
    """Lee un numero entero validando tipo de dato y rango permitido.

    Evita el error tipico de convertir texto no numerico con int(), capturando
    la excepcion ValueError y solicitando nuevamente el dato.

    Parametros:
        mensaje (str): indicacion mostrada al usuario.
        minimo (int): valor minimo aceptado (None = sin limite inferior).
        maximo (int): valor maximo aceptado (None = sin limite superior).
        cancelable(bool): si True, la letra X cancela la operacion.
    Retorna:
        int : numero entero validado, o None si el usuario cancela.
    """
    while True:
        entrada = ""
        try:
            entrada = input(mensaje).strip()
            if cancelable and entrada.upper() == "X":
                return None
            numero = int(entrada) # Puede lanzar ValueError
        except ValueError:
            print(" [!] Dato invalido: '{0}' no es un numero entero.".format(entrada))
            registrar_error("leer_entero", "ValueError con la entrada: " + entrada)
            continue
        except (EOFError, KeyboardInterrupt):
            print("\n [i] Entrada interrumpida por el usuario.")
            return None
        else:
            # El bloque else solo se ejecuta si NO hubo excepcion.
            if minimo is not None and numero < minimo:
                print(" [!] El valor minimo permitido es {0}.".format(minimo))
                continue
            if maximo is not None and numero > maximo:
                print(" [!] El valor maximo permitido es {0}.".format(maximo))

```

```

        continue
    return numero

def leer_decimal(mensaje, minimo=0.0, maximo=None, cancelable=True):
    """Lee un numero decimal (float) validando tipo de dato y rango.

    Acepta coma o punto como separador decimal, error muy frecuente del
    personal administrativo que digita precios.

    Parametros:
        mensaje (str) : indicacion mostrada al usuario.
        minimo (float): valor minimo aceptado.
        maximo (float): valor maximo aceptado (None = sin limite).
        cancelable(bool) : si True, la letra X cancela la operacion.
    Retorna:
        float : numero validado, o None si el usuario cancela.
    """
    while True:
        entrada = ""
        try:
            entrada = input(mensaje).strip().replace(",", ".")
            if cancelable and entrada.upper() == "X":
                return None
            numero = float(entrada) # Puede lanzar ValueError
        except ValueError:
            print(" [!] Dato invalido: ingrese un monto numerico (ej. 59.90).")
            registrar_error("leer_decimal", "ValueError con la entrada: " + entrada)
            continue
        except (EOFError, KeyboardInterrupt):
            print("\n [i] Entrada interrumpida por el usuario.")
            return None
        else:
            if minimo is not None and numero < minimo:
                print(" [!] El monto no puede ser menor que {:.2f}.".format(minimo))
                continue
            if maximo is not None and numero > maximo:
                print(" [!] El monto no puede superar {:.2f}.".format(maximo))
                continue
            return round(numero, 2)

def leer_opcion(mensaje, opciones_validas):
    """Lee una opcion del menu y verifica que pertenezca a las permitidas.

    Parametros:
        mensaje (str) : indicacion mostrada al usuario.
        opciones_validas(list): lista de cadenas con las opciones aceptadas.
    Retorna:
        str : opcion valida elegida por el usuario ("0" si se interrumpe).
    """
    while True:
        try:
            opcion = input(mensaje).strip()
        except (EOFError, KeyboardInterrupt):
            print("\n [i] Sesion interrumpida. Se cerrara el sistema con seguridad.")
            return "0"

        if opcion in opciones_validas:
            return opcion
        print(" [!] Opcion '{0}' no valida. Elija una opcion del menu.".format(opcion))

def confirmar(mensaje):
    """Solicita una confirmacion S/N al usuario.

    Parametros:
        mensaje (str): pregunta mostrada al usuario.
    Retorna:
        bool: True si el usuario responde S, False en cualquier otro caso.
    """
    respuesta = leer_cadena(mensaje + " (S/N): ", obligatorio=True, maximo=3,
                           cancelable=False)
    return respuesta is not None and respuesta.upper() in ("S", "SI")

# =====
# CAPA 4: LOGICA DE NEGOCIO
# =====
# Reglas propias del negocio (busquedas, altas, calculos, reportes).
# IMPORTANTE: ninguna funcion de esta capa usa print() ni input(); todas
# reciben parametros y devuelven valores de retorno, por lo que son
# reutilizables y verificables de forma independiente a la consola.
# =====

def buscar_producto_por_id(inventario, id_producto):
    """Busca un producto por su identificador unico.

    Parametros:
        inventario (list): lista de diccionarios con los productos.
        id_producto (str) : codigo a buscar (no distingue mayusculas).
    Retorna:
        dict : el producto encontrado, o None si no existe.
    """
    if id_producto is None:
        return None
    clave = id_producto.strip().upper()

```

```

    for producto in inventario:
        if producto["id"] == clave:
            return producto
    return None

def existe_id(inventario, id_producto):
    """Indica si un ID ya está registrado (control de duplicados).

    Parametros:
        inventario (list): lista de productos.
        id_producto (str) : código a verificar.
    Retorna:
        bool: True si el ID ya existe en el inventario.
    """
    return buscar_producto_por_id(inventario, id_producto) is not None

def buscar_productos_por_texto(inventario, texto):
    """Busca productos cuyo nombre o categoría contengan el texto indicado.

    Parametros:
        inventario (list): lista de productos.
        texto (str) : palabra o fragmento a buscar.
    Retorna:
        list: productos coincidentes (lista vacía si no hay coincidencias).
    """
    patron = texto.strip().lower()
    coincidencias = []
    for producto in inventario:
        if patron in producto["nombre"].lower() or patron in producto["categoria"].lower():
            coincidencias.append(producto)
    return coincidencias

def generar_id_producto(inventario):
    """Genera automáticamente el siguiente ID correlativo (P001, P002, ...).

    Parametros:
        inventario (list): lista de productos existentes.
    Retorna:
        str: nuevo identificador único sugerido.
    """
    mayor = 0
    for producto in inventario:
        try:
            numero = int(producto["id"][1:])
        except ValueError:
            continue # IDs manuales no correlativos se ignoran
        if numero > mayor:
            mayor = numero
    return "P{0:03d}".format(mayor + 1)

def crear_producto(id_producto, nombre, categoria, precio, stock):
    """Construye y valida el diccionario de un producto.

    Parametros:
        id_producto (str) : código único.
        nombre (str) : descripción comercial.
        categoria (str) : familia a la que pertenece.
        precio (float): precio unitario de venta.
        stock (int) : unidades disponibles.
    Retorna:
        dict: producto listo para incorporarse al inventario.
    Lanza:
        ValueError: si el precio o el stock son negativos, o falta el nombre.
    """
    if not nombre:
        raise ValueError("El nombre del producto es obligatorio")
    if precio < 0:
        raise ValueError("El precio no puede ser negativo")
    if stock < 0:
        raise ValueError("El stock no puede ser negativo")
    return {
        "id": id_producto.strip().upper(),
        "nombre": nombre.strip(),
        "categoria": categoria.strip(),
        "precio": round(float(precio), 2),
        "stock": int(stock),
    }

def agregar_producto(inventario, producto):
    """Da de alta un producto en el inventario controlando IDs duplicados.

    Parametros:
        inventario (list): lista de productos (se modifica en memoria).
        producto (dict): producto validado por crear_producto().
    Retorna:
        bool: True si el alta fue exitosa.
    Lanza:
        ValueError: si el ID ya se encuentra registrado.
    """
    if existe_id(inventario, producto["id"]):
        raise ValueError("El ID {0} ya está registrado".format(producto["id"]))
    inventario.append(producto)

```

```

    return True

def actualizar_precio(inventario, id_producto, nuevo_precio):
    """Modifica el precio unitario de un producto existente.

    Parametros:
        inventario (list) : lista de productos.
        id_producto (str) : codigo del producto a modificar.
        nuevo_precio (float): nuevo precio de venta.
    Retorna:
        dict: el producto actualizado.
    Lanza:
        KeyError : si el producto no existe.
        ValueError: si el precio es negativo.
    """
    producto = buscar_producto_por_id(inventario, id_producto)
    if producto is None:
        raise KeyError("Producto no encontrado: " + str(id_producto))
    if nuevo_precio < 0:
        raise ValueError("El precio no puede ser negativo")
    producto["precio"] = round(float(nuevo_precio), 2)
    return producto

def actualizar_stock(inventario, id_producto, movimiento):
    """Aplica un movimiento de stock (positivo ingreso, negativo salida).

    Parametros:
        inventario (list): lista de productos.
        id_producto (str) : codigo del producto.
        movimiento (int) : unidades a sumar (+) o restar (-).
    Retorna:
        int: stock resultante despues del movimiento.
    Lanza:
        KeyError : si el producto no existe.
        ValueError: si el movimiento dejaria el stock en negativo.
    """
    producto = buscar_producto_por_id(inventario, id_producto)
    if producto is None:
        raise KeyError("Producto no encontrado: " + str(id_producto))
    resultante = producto["stock"] + int(movimiento)
    if resultante < 0:
        raise ValueError("Stock insuficiente: disponible {0}, solicitado {1}".format(
            producto["stock"], abs(movimiento)))
    producto["stock"] = resultante
    return resultante

def eliminar_producto(inventario, id_producto):
    """Da de baja un producto del catalogo.

    Parametros:
        inventario (list): lista de productos.
        id_producto (str) : codigo del producto a eliminar.
    Retorna:
        dict: el producto retirado del inventario.
    Lanza:
        KeyError: si el producto no existe.
    """
    producto = buscar_producto_por_id(inventario, id_producto)
    if producto is None:
        raise KeyError("Producto no encontrado: " + str(id_producto))
    inventario.remove(producto)
    return producto

def calcular_descuento(cantidad):
    """Determina el porcentaje de descuento por volumen de compra.

    Escala comercial vigente:
        12 unidades o mas ... 12 %
        6 a 11 unidades .... 8 %
        3 a 5 unidades .... 5 %
        menos de 3 ..... sin descuento

    Parametros:
        cantidad (int): unidades solicitadas del producto.
    Retorna:
        float: porcentaje de descuento aplicable.
    """
    for minimo, porcentaje in ESCALA_DESCUENTOS:
        if cantidad >= minimo:
            return porcentaje
    return 0.0

def calcular_importe_item(cantidad, precio_unitario, descuento):
    """Calcula el importe neto de una linea de venta.

    Parametros:
        cantidad (int) : unidades vendidas.
        precio_unitario (float): precio de lista.
        descuento (float): porcentaje de descuento aplicado.
    Retorna:
        float: importe con descuento aplicado, redondeado a 2 decimales.
    """

```

```

    bruto = cantidad * precio_unitario
    return round(bruto - (bruto * descuento / 100.0), 2)

def crear_item_venta(producto, cantidad):
    """Construye la línea de detalle de una venta a partir de un producto.

    Parametros:
        producto (dict): producto del inventario.
        cantidad (int) : unidades solicitadas.
    Retorna:
        dict: item con cantidad, precio, descuento e importe.
    Lanza:
        ValueError: si la cantidad no es positiva o supera el stock.
    """
    if cantidad <= 0:
        raise ValueError("La cantidad debe ser mayor que cero")
    if cantidad > producto["stock"]:
        raise ValueError("Stock insuficiente: solo quedan {0} unidad(es)".format(
            producto["stock"]))
    descuento = calcular_descuento(cantidad)
    return {
        "id": producto["id"],
        "nombre": producto["nombre"],
        "cantidad": cantidad,
        "precio_unitario": producto["precio"],
        "descuento": descuento,
        "importe": calcular_importe_item(cantidad, producto["precio"], descuento),
    }

def generar_numero_boleta(ventas):
    """Genera el número correlativo de la siguiente boleta (B0001, B0002, ...).

    Parametros:
        ventas (list): historial de ventas registradas.
    Retorna:
        str: número de boleta correlativo.
    """
    return "B{0:04d}".format(len(ventas) + 1)

def construir_venta(numero_boleta, cliente, items):
    """Arma el comprobante de venta con sus totales e impuestos.

    Estructura resultante (diccionario anidado):
        {boleta, fecha, cliente, items:[{...}, {...}], subtotal, igv, total}

    Parametros:
        numero_boleta (str) : correlativo del comprobante.
        cliente (str) : nombre del cliente.
        items (list): líneas de detalle de la venta.
    Retorna:
        dict: comprobante de venta completo.
    Lanza:
        ValueError: si la venta no contiene items.
    """
    if not items:
        raise ValueError("La venta debe contener al menos un producto")
    subtotal = round(sum(item["importe"] for item in items), 2)
    igv = round(subtotal * IGv, 2)
    return {
        "boleta": numero_boleta,
        "fecha": datetime.now().strftime("%d/%m/%Y %H:%M:%S"),
        "cliente": cliente if cliente else "CLIENTE VARIOS",
        "items": items,
        "subtotal": subtotal,
        "igv": igv,
        "total": round(subtotal + igv, 2),
    }

def descontar_stock_de_venta(inventario, items):
    """Descuenta del inventario las unidades vendidas de cada item.

    Parametros:
        inventario (list): lista de productos.
        items (list): líneas de detalle de la venta.
    Retorna:
        int: cantidad de productos cuyo stock fue actualizado.
    """
    actualizados = 0
    for item in items:
        actualizar_stock(inventario, item["id"], -item["cantidad"])
        actualizados += 1
    return actualizados

def valorizar_inventario(inventario):
    """Calcula el valor total del inventario a precio de venta.

    Parametros:
        inventario (list): lista de productos.
    Retorna:
        float: sumatoria de precio * stock de todos los productos.
    """
    return round(sum(p["precio"] * p["stock"] for p in inventario), 2)

```



```

def productos_stock_critico(inventario, minimo=STOCK_MINIMO):
    """Devuelve los productos que requieren reposición urgente.

    Parametros:
        inventario (list): lista de productos.
        minimo (int) : umbral de stock critico.
    Retorna:
        list: productos con stock menor o igual al umbral.
    """
    return [p for p in inventario if p["stock"] <= minimo]

def resumen_ventas(ventas, fecha=None):
    """Consolida los indicadores del historial de ventas.

    Parametros:
        ventas (list): historial de ventas.
        fecha (str) : dia a filtrar con formato dd/mm/aaaa (None = todo).
    Retorna:
        dict: {cantidad, subtotal, igv, total, ticket_promedio, unidades}
    """
    seleccion = ventas
    if fecha is not None:
        seleccion = [v for v in ventas if v["fecha"].startswith(fecha)]

    total = round(sum(v["total"] for v in seleccion), 2)
    unidades = sum(item["cantidad"] for v in seleccion for item in v["items"])
    cantidad = len(seleccion)
    return {
        "cantidad": cantidad,
        "subtotal": round(sum(v["subtotal"] for v in seleccion), 2),
        "igv": round(sum(v["igv"] for v in seleccion), 2),
        "total": total,
        "unidades": unidades,
        "ticket_promedio": round(total / cantidad, 2) if cantidad else 0.0,
    }

def ranking_productos(ventas, top=5):
    """Determina los productos mas vendidos del historial.

    Parametros:
        ventas (list): historial de ventas.
        top (int) : cantidad de posiciones a devolver.
    Retorna:
        list: tuplas (id, unidades vendidas) ordenadas de mayor a menor.
    """
    acumulado = {} # Diccionario id -> unidades
    for venta in ventas:
        for item in venta["items"]:
            acumulado[item["id"]] = acumulado.get(item["id"], 0) + item["cantidad"]
    ordenado = sorted(acumulado.items(), key=lambda par: par[1], reverse=True)
    return ordenado[:top]

# =====
# CAPA 5: INTERFAZ DE CONSOLA
# -----
# Unico lugar del programa donde se usan print() e input() para dialogar con el
# usuario. Se apoya en la CAPA 3 para validar y en la CAPA 4 para calcular.
# =====

def limpiar_pantalla():
    """Limpia la consola segun el sistema operativo (Windows / Linux / Mac)."""
    try:
        os.system("cls" if os.name == "nt" else "clear")
    except OSError:
        print("\n" * 3)

def mostrar_encabezado(titulo):
    """Imprime el encabezado estandar de cada modulo del sistema.

    Parametros:
        titulo (str): nombre de la operacion en curso.
    """
    print("=" * ANCHO)
    print(" {0}".format(NOMBRE_EMPRESA).ljust(ANCHO - 24) +
          datetime.now().strftime("%d/%m/%Y %H:%M"))
    print(" {0}".format(titulo.upper()))
    print("=" * ANCHO)

def mostrar_menu_principal():
    """Muestra el menu principal con todas las operaciones disponibles."""
    print("=" * ANCHO)
    print("{0:^{1}}".format("SISTEMA DE GESTION DE INVENTARIO Y VENTAS", ANCHO))
    print("{0:^{1}}".format(NOMBRE_EMPRESA + " - " + RUBRO_EMPRESA, ANCHO))
    print("=" * ANCHO)
    print("    GESTION DE INVENTARIO                VENTAS Y REPORTES")
    print("    -----")
    print("    [1] Listar inventario                [5] Registrar venta")
    print("    [2] Registrar nuevo producto        [6] Historial de ventas")
    print("    [3] Buscar producto                  [7] Reporte de inventario")
    print("    [4] Actualizar producto              [8] Reporte de ventas")

```

```

        print("                                [9] Guardar cambios")
        print("    [0] Salir del sistema")
        print("=" * ANCHO)

def mostrar_tabla_productos(productos):
    """Presenta una lista de productos en formato de tabla alineada.

    Parametros:
        productos (list): productos a mostrar.
    Retorna:
        int: cantidad de productos mostrados.
    """
    if not productos:
        print("    [i] No hay productos que mostrar.")
        return 0

    print("-" * ANCHO)
    print(" {0:<6} {1:<26} {2:<14} {3:>10} {4:>7} {5:>9}".format(
        "ID", "PRODUCTO", "CATEGORIA", "PRECIO S/", "STOCK", "ESTADO"))
    print("-" * ANCHO)
    for producto in productos:
        if producto["stock"] == 0:
            estado = "AGOTADO"
        elif producto["stock"] <= STOCK_MINIMO:
            estado = "CRITICO"
        else:
            estado = "OK"
        print(" {0:<6} {1:<26} {2:<14} {3:>10.2f} {4:>7} {5:>9}".format(
            producto["id"], producto["nombre"][:26], producto["categoria"][:14],
            producto["precio"], producto["stock"], estado))
    print("-" * ANCHO)
    print(" Total de productos listados: {0}".format(len(productos)))
    return len(productos)

def mostrar_comprobante(venta):
    """Imprime en pantalla el comprobante (boleta) de una venta.

    Parametros:
        venta (dict): comprobante generado por construir_venta().
    """
    print()
    print("+ " + "-" * (ANCHO - 2) + "+")
    print("|{0:^{1}}|".format(NOMBRE_EMPRESA, ANCHO - 2))
    print("|{0:^{1}}|".format("BOLETA DE VENTA ELECTRONICA " + venta["boleta"], ANCHO - 2))
    print("+ " + "-" * (ANCHO - 2) + "+")
    print(" Fecha : {0}".format(venta["fecha"]))
    print(" Cliente: {0}".format(venta["cliente"]))
    print("-" * ANCHO)
    print(" {0:<6} {1:<24} {2:>5} {3:>10} {4:>7} {5:>11}".format(
        "ID", "DESCRIPCION", "CANT", "P.UNIT", "DSCTO%", "IMPORTE"))
    print("-" * ANCHO)
    for item in venta["items"]:
        print(" {0:<6} {1:<24} {2:>5} {3:>10.2f} {4:>7.1f} {5:>11.2f}".format(
            item["id"], item.get("nombre", "")[:24], item["cantidad"],
            item["precio_unitario"], item["descuento"], item["importe"]))
    print("-" * ANCHO)
    print("{0:>66} {1:>11.2f}".format("SUBTOTAL S/", venta["subtotal"]))
    print("{0:>66} {1:>11.2f}".format("IGV (18%) S/", venta["igv"]))
    print("{0:>66} {1:>11.2f}".format("TOTAL S/", venta["total"]))
    print("=" * ANCHO)

def pausar():
    """Detiene la ejecucion hasta que el usuario presione ENTER."""
    try:
        input("\n Presione ENTER para volver al menu principal...")
    except (EOFError, KeyboardInterrupt):
        print()

# -----
# OPERACIONES DEL MENU (una funcion por cada opcion: separacion clara)
# -----

def opcion_listar_inventario(inventario):
    """Opción 1: muestra todo el catalogo de productos.

    Parametros:
        inventario (list): lista de productos en memoria.
    """
    mostrar_encabezado("Listado general del inventario")
    mostrar_tabla_productos(inventario)
    if inventario:
        print(" Valorizado total del inventario: S/ {0:,.2f}".format(
            valorizar_inventario(inventario)))

def opcion_registrar_producto(inventario):
    """Opción 2: da de alta un producto nuevo en el inventario.

    Valida el ID duplicado, el precio y el stock antes de aceptar el registro.

    Parametros:
        inventario (list): lista de productos (se modifica en memoria).
    Retorna:

```

```

        bool: True si el producto fue registrado.
    """
    mostrar_encabezado("Registro de nuevo producto")
    print("- (Escriba X en cualquier momento para cancelar la operacion)\n")

    sugerido = generar_id_producto(inventario)
    id_producto = leer_cadena(" ID del producto [ENTER = {0}]: ".format(sugerido),
                              obligatorio=False, maximo=6)

    if id_producto is None:
        print("\n [i] Operacion cancelada por el usuario.")
        return False
    if not id_producto:
        id_producto = sugerido

    if existe_id(inventario, id_producto):
        print(" [!] El ID '{0}' ya existe. No se permiten duplicados.".format(
            id_producto.upper()))
        return False

    nombre = leer_cadena(" Nombre del producto : ", maximo=26)
    if nombre is None:
        print("\n [i] Operacion cancelada por el usuario.")
        return False

    print(" Categorías disponibles: " + ", ".join(
        "{0}={1}".format(i + 1, c) for i, c in enumerate(CATEGORIAS)))
    indice = leer_entero(" Categoría (1-{0}) : ".format(len(CATEGORIAS)),
                        minimo=1, maximo=len(CATEGORIAS))

    if indice is None:
        print("\n [i] Operacion cancelada por el usuario.")
        return False
    categoria = CATEGORIAS[indice - 1]

    precio = leer_decimal(" Precio unitario de venta S/ : ", minimo=0.10, maximo=99999.0)
    if precio is None:
        print("\n [i] Operacion cancelada por el usuario.")
        return False

    stock = leer_entero(" Stock inicial (unidades) : ", minimo=0, maximo=99999)
    if stock is None:
        print("\n [i] Operacion cancelada por el usuario.")
        return False

    registrado = False
    try:
        producto = crear_producto(id_producto, nombre, categoria, precio, stock)
        agregar_producto(inventario, producto)
    except ValueError as error:
        print("\n [!] No se pudo registrar el producto: {0}".format(error))
        registrar_error("opcion_registrar_producto", str(error))
    else:
        registrado = True
        print("\n [OK] Producto '{0}' registrado con el código {1}".format(
            producto["nombre"], producto["id"]))
        guardar_inventario(inventario) # Persistencia inmediata
    finally:
        print(" [i] Fin del proceso de alta de producto.")
    return registrado

def opcion_buscar_producto(inventario):
    """Opción 3: Busca productos por código o por texto libre.

    Parametros:
        inventario (list): lista de productos.
    """
    mostrar_encabezado("Busqueda de productos")
    if not inventario:
        print(" [i] El inventario esta vacio. Registre productos primero.")
        return

    print(" [1] Buscar por ID exacto")
    print(" [2] Buscar por nombre o categoria")
    tipo = leer_opcion("\n Seleccione el tipo de busqueda: ", ["1", "2"])

    if tipo == "1":
        id_producto = leer_cadena(" Ingrese el ID a consultar: ", maximo=6)
        if id_producto is None:
            return
        producto = buscar_producto_por_id(inventario, id_producto)
        if producto is None:
            print("\n [!] No existe un producto con el código '{0}'.".format(
                id_producto.upper()))
        else:
            print()
            mostrar_tabla_productos([producto])
            print(" Valorizado de este producto: S/ {0:,.2f}".format(
                producto["precio"] * producto["stock"]))
    else:
        texto = leer_cadena(" Ingrese el texto a buscar: ", maximo=26)
        if texto is None:
            return
        coincidencias = buscar_productos_por_texto(inventario, texto)
        print()
        if not coincidencias:
            print(" [!] Sin coincidencias para '{0}'.".format(texto))
        else:

```

```

        mostrar_tabla_productos(coincidencias)

def opcion_actualizar_producto(inventario):
    """Opción 4: modifica precio, stock o da de baja un producto.

    Parametros:
        inventario (list): lista de productos.
    Retorna:
        bool: True si se realizo alguna modificacion.
    """
    mostrar_encabezado("Actualizacion de productos")
    if not inventario:
        print("\n [i] El inventario esta vacio. Registre productos primero.")
        return False

    id_producto = leer_cadena(" ID del producto a actualizar (X = cancelar): ", maximo=6)
    if id_producto is None:
        print("\n [i] Operacion cancelada por el usuario.")
        return False

    producto = buscar_producto_por_id(inventario, id_producto)
    if producto is None:
        print("\n [!] No existe un producto con el codigo '{0}'.".format(
            id_producto.upper()))
        return False

    print()
    mostrar_tabla_productos([producto])
    print("\n [1] Modificar precio de venta")
    print("\n [2] Registrar ingreso de mercaderia (sumar stock)")
    print("\n [3] Registrar merma o ajuste (restar stock)")
    print("\n [4] Dar de baja el producto")
    accion = leer_opcion("\n Seleccione la accion: ", ["1", "2", "3", "4"])

    modificado = False
    try:
        if accion == "1":
            nuevo = leer_decimal(" Nuevo precio S/ : ", minimo=0.10, maximo=99999.0)
            if nuevo is None:
                raise ValueError("Operacion cancelada por el usuario")
            anterior = producto["precio"]
            actualizar_precio(inventario, producto["id"], nuevo)
            print("\n [OK] Precio actualizado: S/ {0:.2f} -> S/ {1:.2f}".format(
                anterior, producto["precio"]))
        elif accion == "2":
            unidades = leer_entero(" Unidades que ingresan: ", minimo=1, maximo=9999)
            if unidades is None:
                raise ValueError("Operacion cancelada por el usuario")
            resultante = actualizar_stock(inventario, producto["id"], unidades)
            print("\n [OK] Stock actualizado. Nuevo stock: {0} unidad(es)".format(resultante))
        elif accion == "3":
            unidades = leer_entero(" Unidades que salen: ", minimo=1, maximo=9999)
            if unidades is None:
                raise ValueError("Operacion cancelada por el usuario")
            resultante = actualizar_stock(inventario, producto["id"], -unidades)
            print("\n [OK] Stock actualizado. Nuevo stock: {0} unidad(es)".format(resultante))
        else:
            if not confirmar("\n Confirma dar de baja '{0}'?".format(producto["nombre"])):
                raise ValueError("Baja no confirmada por el usuario")
            eliminado = eliminar_producto(inventario, producto["id"])
            print("\n [OK] Producto '{0}' dado de baja del catalogo.".format(
                eliminado["nombre"]))
    except ValueError as error:
        print("\n [!] Operacion no realizada: {0}".format(error))
        registrar_error("opcion_actualizar_producto", str(error))
    except KeyError as error:
        print("\n [!] Producto no encontrado: {0}".format(error))
        registrar_error("opcion_actualizar_producto", str(error))
    else:
        modificado = True
        guardar_inventario(inventario) # Persistencia inmediata
    finally:
        print("\n [i] Fin del proceso de actualizacion.")
    return modificado

def opcion_registrar_venta(inventario, ventas):
    """Opción 5: registra una transaccion de venta con varios productos.

    Flujo: se agregan items en un bucle while (break para terminar, continue
    ante errores), se calculan descuentos e IGV, se descuenta el stock y se
    graba la venta en el historial (modo "a") y el inventario (modo "w").

    Parametros:
        inventario (list): lista de productos.
        ventas (list): historial de ventas en memoria.
    Retorna:
        bool: True si la venta se registro correctamente.
    """
    mostrar_encabezado("Registro de venta")
    disponibles = [p for p in inventario if p["stock"] > 0]
    if not disponibles:
        print("\n [!] No hay productos con stock disponible para vender.")
        return False

    mostrar_tabla_productos(disponibles)

```

```

cliente = leer_cadena("\n Nombre del cliente [ENTER = CLIENTE VARIOS]: ",
                     obligatorio=False, maximo=30)
if cliente is None:
    print("\n [i] Venta cancelada por el usuario.")
    return False

items = []
print("\n Agregue los productos de la venta (escriba F para finalizar).")
while True:
    id_producto = leer_cadena("\n ID del producto (F = finalizar): ",
                              maximo=6, cancelable=False)
    if id_producto is None or id_producto.upper() == "F":
        break
    # Termina la carga de items

    producto = buscar_producto_por_id(inventario, id_producto)
    if producto is None:
        print(" [!] El producto '{0}' no existe en el catalogo.".format(
            id_producto.upper()))
        continue
    # Vuelve a pedir el ID
    if producto["stock"] <= 0:
        print(" [!] '{0}' se encuentra AGOTADO.".format(producto["nombre"]))
        continue

    cantidad = leer_entero(" Cantidad (stock disponible {0}): ".format(
        producto["stock"]), minimo=1, maximo=99999)
    if cantidad is None:
        print(" [i] Item descartado.")
        continue

    try:
        item = crear_item_venta(producto, cantidad)
    except ValueError as error:
        print(" [!] {0}".format(error))
        registrar_error("opcion_registrar_venta", str(error))
        continue
    # El item no se agrega, la venta sigue
    else:
        items.append(item)
        print(" [OK] Agregado: {0} x {1} = S/ {2:.2f} (dscto {3:.1f}%)".format(
            cantidad, item["nombre"], item["importe"], item["descuento"]))

if not items:
    print("\n [i] La venta fue anulada: no se agregaron productos.")
    return False

registrada = False
try:
    venta = construir_venta(generar_numero_boleta(ventas), cliente, items)
    descontar_stock_venta(inventario, items)
except ValueError as error:
    print("\n [!] No se pudo procesar la venta: {0}".format(error))
    registrar_error("opcion_registrar_venta", str(error))
except KeyError as error:
    print("\n [!] Error de integridad del inventario: {0}".format(error))
    registrar_error("opcion_registrar_venta", str(error))
else:
    ventas.append(venta)
    mostrar_comprobante(venta)
    registrar_venta_en_archivo(venta) # Modo "a": agrega al historial
    guardar_inventario(inventario) # Modo "w": stock actualizado
    registrada = True
finally:
    print(" [i] Transaccion finalizada.")
    return registrada

def opcion_historial_ventas(ventas):
    """Opción 6: lista las ventas registradas en el historial.

    Parametros:
        ventas (list): historial de ventas.
    """
    mostrar_encabezado("Historial de ventas")
    if not ventas:
        print(" [i] Aun no se han registrado ventas.")
        return

    print("-" * ANCHO)
    print(" {0:<8} {1:<20} {2:<24} {3:>6} {4:>12}".format(
        "BOLETA", "FECHA", "CLIENTE", "ITEMS", "TOTAL S/"))
    print("-" * ANCHO)
    for venta in ventas:
        print(" {0:<8} {1:<20} {2:<24} {3:>6} {4:>12.2f}".format(
            venta["boleto"], venta["fecha"], venta["cliente"][:24],
            len(venta["items"]), venta["total"]))
    print("-" * ANCHO)
    resumen = resumen_ventas(ventas)
    print(" Comprobantes: {0} Unidades vendidas: {1} Recaudado: S/ {2:.2f}".format(
        resumen["cantidad"], resumen["unidades"], resumen["total"]))

def opcion_reporte_inventario(inventario):
    """Opción 7: genera el reporte de situacion del inventario.

    Parametros:
        inventario (list): lista de productos.
    """

```

```

mostrar_encabezado("Reporte de inventario")
if not inventario:
    print(" [i] El inventario esta vacio.")
    return

unidades = sum(p["stock"] for p in inventario)
criticos = productos_stock_critico(inventario)
agotados = [p for p in inventario if p["stock"] == 0]

print(" Productos registrados .....: {0}".format(len(inventario)))
print(" Unidades totales en stock ....: {0}".format(unidades))
print(" Valorizado del inventario ....: S/ {0:,.2f}".format(
    valorizar_inventario(inventario)))
print(" Productos en stock critico ..: {0} (umbral {1} unidades)".format(
    len(criticos), STOCK_MINIMO))
print(" Productos agotados .....: {0}".format(len(agotados)))

print("\n RESUMEN POR CATEGORIA")
print("-" * ANCHO)
print(" {0:<16} {1:>10} {2:>12} {3:>16}".format(
    "CATEGORIA", "PRODUCTOS", "UNIDADES", "VALORIZADO S/"))
print("-" * ANCHO)
for categoria in CATEGORIAS:
    grupo = [p for p in inventario if p["categoria"] == categoria]
    if not grupo:
        continue
    print(" {0:<16} {1:>10} {2:>12} {3:>16,.2f}".format(
        categoria, len(grupo), sum(p["stock"] for p in grupo),
        valorizar_inventario(grupo)))
print("-" * ANCHO)

if criticos:
    print("\n ALERTA: PRODUCTOS QUE REQUIEREN REPOSICION")
    mostrar_tabla_productos(criticos)

def opcion_reporte_ventas(ventas):
    """Opción 8: genera el reporte de rendimiento de ventas.

    Parametros:
        ventas (list): historial de ventas.
    """
    mostrar_encabezado("Reporte de ventas")
    if not ventas:
        print(" [i] Aun no se han registrado ventas.")
        return

    hoy = datetime.now().strftime("%d/%m/%Y")
    del_dia = resumen_ventas(ventas, hoy)
    historico = resumen_ventas(ventas)

    print(" VENTAS DEL DIA {0}".format(hoy))
    print("-" * ANCHO)
    print(" Comprobantes emitidos .....: {0}".format(del_dia["cantidad"]))
    print(" Unidades vendidas .....: {0}".format(del_dia["unidades"]))
    print(" Venta neta (sin IGV) .....: S/ {0:,.2f}".format(del_dia["subtotal"]))
    print(" IGV del dia .....: S/ {0:,.2f}".format(del_dia["igv"]))
    print(" Total recaudado .....: S/ {0:,.2f}".format(del_dia["total"]))
    print(" Ticket promedio .....: S/ {0:,.2f}".format(del_dia["ticket_promedio"]))

    print("\n ACUMULADO HISTORICO")
    print("-" * ANCHO)
    print(" Comprobantes .....: {0}".format(historico["cantidad"]))
    print(" Total recaudado .....: S/ {0:,.2f}".format(historico["total"]))
    print(" Ticket promedio .....: S/ {0:,.2f}".format(historico["ticket_promedio"]))

    ranking = ranking_productos(ventas)
    if ranking:
        print("\n PRODUCTOS MAS VENDIDOS")
        print("-" * ANCHO)
        print(" {0:<8} {1:<30} {2:>12}".format("ID", "PRODUCTO", "UNIDADES"))
        print("-" * ANCHO)
        for posicion, (id_producto, unidades) in enumerate(ranking, start=1):
            nombre = "(producto retirado del catalogo)"
            for venta in ventas:
                for item in venta["items"]:
                    if item["id"] == id_producto and item.get("nombre"):
                        nombre = item["nombre"]
                        break
            print(" {0:<8} {1:<30} {2:>12}".format(id_producto, nombre[:30], unidades))
        print("-" * ANCHO)

def opcion_guardar_cambios(inventario):
    """Opción 9: fuerza el guardado del inventario en el archivo de texto.

    Parametros:
        inventario (list): lista de productos.
    Retorna:
        bool: True si el guardado fue exitoso.
    """
    mostrar_encabezado("Guardado de informacion")
    resultado = guardar_inventario(inventario)
    if resultado:
        print("\n [OK] La informacion quedara disponible en la proxima ejecucion.")
    else:
        print("\n [!] Revise el archivo datos/errores.log para mayor detalle.")

```

```

    return resultado

# =====
# CONTROLADOR PRINCIPAL
# =====
# Bucle while que mantiene el menu activo e if-elif-else que deriva cada
# opcion hacia su funcion especializada. Es la unica funcion que coordina; no
# contiene reglas de negocio ni accesos directos a disco.
# =====

def cargar_datos_iniciales():
    """Carga inventario e historial desde los archivos al iniciar el sistema.

    Retorna:
        tuple: (inventario, ventas) como listas de diccionarios.
    """
    mostrar_encabezado("Iniciando el sistema")
    inicializar_almacenamiento()
    inventario = cargar_inventario()
    ventas = cargar_ventas()
    print("-" * ANCHO)
    return inventario, ventas

def main():
    """Punto de entrada del sistema: menu principal persistente.

    Estructura de control:
        while True ..... mantiene el menu activo hasta elegir salir.
        if - elif - else ..... deriva la opcion hacia su funcion modular.
        break ..... unica salida controlada del bucle.
    """
    inventario, ventas = cargar_datos_iniciales()
    pausar()

    while True:
        limpiar_pantalla()
        mostrar_menu_principal()
        opcion = leer_opcion(" Seleccione una opcion [0-9]: ", OPCIONES_MENU)
        print()

        if opcion == "1":
            opcion_listar_inventario(inventario)
        elif opcion == "2":
            opcion_registrar_producto(inventario)
        elif opcion == "3":
            opcion_buscar_producto(inventario)
        elif opcion == "4":
            opcion_actualizar_producto(inventario)
        elif opcion == "5":
            opcion_registrar_venta(inventario, ventas)
        elif opcion == "6":
            opcion_historial_ventas(ventas)
        elif opcion == "7":
            opcion_reporte_inventario(inventario)
        elif opcion == "8":
            opcion_reporte_ventas(ventas)
        elif opcion == "9":
            opcion_guardar_cambios(inventario)
        else:
            # Opcion "0": salida controlada con guardado de seguridad.
            mostrar_encabezado("Cierre del sistema")
            guardar_inventario(inventario)
            print("\n Resumen de la sesion:")
            print("  - Productos en catalogo : {}".format(len(inventario)))
            print("  - Ventas registradas      : {}".format(len(ventas)))
            print("  - Valorizado final        : S/ {}".format(
                valorizar_inventario(inventario)))
            print("\n Gracias por utilizar el sistema. Hasta pronto!")
            print("=" * ANCHO)
            break
            # Unica salida del bucle principal

    pausar()

# =====
# ARRANQUE DEL PROGRAMA
# =====
# La condicion __name__ == "__main__" permite que el archivo se ejecute como
# programa y, a la vez, que sus funciones puedan importarse desde otro modulo
# sin que el menu se dispare automaticamente.
# =====

if __name__ == "__main__":
    try:
        main()
    except KeyboardInterrupt:
        print("\n\n [i] Ejecucion interrumpida con Ctrl+C. Cierre seguro del sistema.")
    except Exception as error:
        # Red de seguridad de ultimo nivel
        print("\n [!] Error inesperado: {}".format(error))
        registrar_error("main", "{}: {}".format(type(error).__name__, error))
        print(" [i] El detalle tecnico se registro en '{}'.format(ARCHIVO_LOG))
    finally:
        print(" [i] Proceso finalizado.Codigo de salida 0.")
        sys.exit(0)

```


ANEXO B. ARCHIVOS DE DATOS GENERADOS

Contenido de la carpeta `datos/` al finalizar la sesión documentada en las capturas de pantalla. Estos archivos son los que garantizan la persistencia de la información entre ejecuciones.

B.1. `datos/inventario.txt` (modo de escritura "w")

```
P001;Audifonos Bluetooth ANC;Audio;159.90;14
P002;Mouse Inalambrico 2.4G;Computo;49.90;16
P003;Teclado Mecanico RGB;Computo;229.00;8
P004;Cable HDMI 2.1 - 2m;Conectividad;35.50;28
P005;Power Bank 20000 mAh;Energia;129.00;4
P006;Memoria USB 128 GB;Computo;69.90;17
P007;Hub USB-C 7 en 1;Conectividad;119.00;6
P008;Cargador GaN 65W;Energia;149.00;22
P009;Parlante Bluetooth 20W;Audio;189.00;9
P010;Soporte Laptop Aluminio;Accesorios;89.90;12
P011;Webcam Full HD 1080p;Computo;189.50;14
```

Formato: id;nombre;categoria;precio;stock. El stock refleja los descuentos aplicados por la venta registrada.

B.2. `datos/ventas.txt` (modo de agregado "a")

```
B0001;12/09/2026 00:00:06;Maria Quispe Ruiz;P002:6:49.90:8.0|P004:12:35.50:12.0|
P011:1:189.50:0.0;839.83;151.17;991.00
```

Formato: boleta;fecha;cliente;items;subtotal;igv;total, donde cada ítem es id:cantidad:precio:descuento.

B.3. `datos/errores.log` (modo de agregado "a")

```
[11/09/2026 23:59:58] leer_decimal -> ValueError con la entrada: ciento ochenta
[12/09/2026 00:00:04] leer_entero -> ValueError con la entrada: seis
[12/09/2026 00:00:05] opcion_registrar_venta -> Stock insuficiente: solo quedan 40 unidad(es)
```

Bitácora de las excepciones capturadas durante la sesión: ninguna de ellas detuvo la ejecución del programa.